CROSSMATCH

DIGITALPERSONA

U.ARE.U® SDK 2.3

DEVELOPER GUIDE

Copyright © 2015 Crossmatch. All rights reserved. Specifications are subject to change without prior notice. The Crossmatch logo, Crossmatch™, Cross Match®, L Scan®, D Scan®, I Scan®, Guardian®, SEEK® and Verifier® are trademarks or registered trademarks of Cross Match Technologies, Inc. in the United States and other countries. DigitalPersona®, TouchChip®, Eikon®, U.are.U® and FingerJet™ are trademarks or registered trademarks of DigitalPersona, Inc., which is owned by the parent company of Cross Match Technologies, Inc. All other brand and product names are trademarks or registered trademarks of their respective owners.

Published: April 8, 2015 (v2.3.0)

Contents

INTRODUCTION 7

What's New in 2.3.x 7
How the U.are.U Manuals are Organized 7
Getting Updated Documentation 8
Target Audience 8
How this book is arranged 8
Data Formats and Standards 8

BACKGROUND 9

Biometrics 9
Features of Biometric Technology 9
The Basics of Fingerprint Identification 9
Issues in Fingerprint Recognition Technology 10

DEVELOPING APPLICATIONS 11

How Fingerprint Recognition Works 11
Understanding the Data Flow 11
Workflow - Enrollment Application 12
  Step One - Initialization 12
  Step Two - Capture, Extract and Enroll 12
  Step Three - Store Data 13
  Notes on Enrollment 13
Workflow - Identifying/Verifying 13
  Step One - Initialization 13
  Step Two - Capture and Extract 13
  Step Three - Identify/Verify 13
Design Issues for your Application 13
  Distributed Processing and Data Flow 13
  Data Compression 14
    WSQ Compression 14
    JPEG2000 Compression 14
  NIST Fingerprint Image Quality (NFIQ) 14
  Determining an Acceptable Level of Error 15
    Setting the Error Threshold when Identifying a Fingerprint in a Collection 15
Defining the Data Retention Policy 16
  Specifying the Fingers to Be Scanned 16
  Optimizing Fingerprint Applications 16

WORKING WITH FINGERPRINT READERS 17

UPGRADING FROM PREVIOUS SDKS 18

Overview of Support for Previous SDKs 18
  Working with Existing Data Created with Previous SDKs 18
  Exchanging Data with Applications Using Previous SDKs 18
  Upgrading Applications from Previous SDKs 18
Converting Applications from the One Touch SDK 18
Working with Data Created with Gold and One Touch SDKs 19
Changes in U.are.U Terminology from Previous Usage (Gold and One Touch) 20
Upgrading from U.are.U UPOS for OPOS/JavaPOS 20

USING THE SDK 22
What's In the SDK? ... 22
FingerJet Engine ... 22

THE C/C++ APIS 23
DP Capture API ... 23
Library Management ... 23
Fingerprint Capture Device Management ... 23
Capturing Fingerprints ... 24
Streaming Fingerprints ... 24
FingerJet Engine API ... 24
Library Management ... 24
Select Fingerprint Matching Engine ... 24
Extract FMD ... 25
Identify Fingerprint ... 25
Enrollment ... 25
Format Conversion ... 25
Advanced Diagnostics ... 26
Wavelet Scalar Quantization (WSQ) Compression ... 26
NIST Fingerprint Image Quality (NFIQ) ... 27

THE JAVA API 28
Importing the U.are.U Java package ... 28
Getting Detailed Documentation ... 28
Using the Package ... 28
Main Access Point ... 28
UareUException ... 28
Getting a List of Available Readers ... 28
Working with Readers ... 29
Capturing Fingerprints ... 29
Streaming Fingerprints ... 30
Accessing the FingerJet Engine ... 30
Creating FMDs from images ... 30
Identification and Comparison ... 30
Enrollment ... 31

THE .NET API 32
Importing the U.are.U .NET package ... 32
Getting Detailed Documentation ... 32
Using the Package ... 32
Main Access Points ... 32
SDKException ... 32
Serialization ... 33
IEnumerables in the .NET Wrapper ... 33
Working with Readers ... 33
Capturing Fingerprints ... 35
Streaming Fingerprints ... 35
Managing Fingerprint Data ... 36
Analyzing and Managing Fingerprints (FingerJet Engine) ... 37
Creating FMDs from images ... 38
Comparing Fingerprints ... 38

Enrollment 39
Common Data Structures for Results 40
Pre-Built Controls for Enrollment and Identification 41

ACTIVEX 42

Importing the U.are.U ActiveX package 42
Getting Detailed Documentation 42
Using the Package 42
Main Access Points 42
SDKException 42
Serialization 42
Working with Readers 43
A Note About Internet Explorer and Process Merging 43
Class Diagrams 44
Capturing Fingerprints 46
Streaming Fingerprints 46
Managing Fingerprint Data 47
Accessing the FingerJet Engine 48
Creating FMDs from images 48
Identification and Comparison 49
Enrollment 49
Common Data Structures for Results 50
Pre-Built Controls for Enrollment and Identification 51

JAVAPOS 52

Terminology Note 52
Working with Fingerprint Data in JavaPOS 52
Fingerprint Data for Raw Images (Captures) 52
Fingerprint Data for Captures and Enrollment Templates (BIRs) 52
Working with DigitalPersona Record Formats 53
Converting to a Different Data Format 53
Getting Device-Specific Information with DirectIOEvent 54
Syntax 54
Properties 54
EventNumber Return Values 54
Implementation Notes 55
Exceptions 57
Device-Related Error Codes 57

OPOS 59

Terminology Note 59
Working with Fingerprint Data in OPOS 59
Fingerprint Data for Raw Images (Captures) 59
Fingerprint Data for Captures and Enrollment Templates (BIRs) 59
Working with DigitalPersona Record Formats 60
Converting to a Different Data Format 60
Getting Device-Specific Information with DirectIOEvent 61
Syntax 61
Description 61
Attribute 61
Syntax 61

Description 61
Attributes 61
EventNumber Return Values 62
Data Return Values 62
Obj Return Values 62
Syntax 63
Description 63
Attribute 63
Status Return Values 63
Implementation Notes 63
Exceptions 66
Device-Related Error Codes 67

APPLICATION NOTES 68

General Fingerprint Issues 68
Minex Certification 68
C/C++ Issues 68

GLOSSARY 69

General Terms 69
Fingerprint Data 69
Fingerprint Devices 70
Fingerprint Recognition Terms 70
Recognition Accuracy 71

INDEX 73

Introduction

THIS CHAPTER PROVIDES AN OVERVIEW OF THE FUNCTIONALITY AND FEATURES OF THE U.ARE.U SDK

The U.are.U SDK allows you to add fingerprint recognition to your application. Significant features of the U.are.U SDK include:

- Performing 1-to-many fingerprint identification automatically
- Support for multiple data formats, including the existing DigitalPersona data formats as well as ANSI and ISO fingerprint images and minutiae data
- Redesigned API for easier programming
- New FingerJet Engine that has met the PIV performance thresholds for fingerprint minutiae data generation required by NIST, with a faster Matcher.

The SDK provides support for developing with Windows (Visual Studio) and Linux with the following languages/ frameworks: (X indicates a supported combination)

API	Linux	Windows	Windows CE	Android
C/C++	X	X	X	X
Java	X	X		X
.NET		X	X	
ActiveX		X		
JavaPOS	X	X		
OPOS		X		

The U.are.U SDK supports:

- Development on Windows for target devices based on Windows or Windows CE
- Development on Linux for target devices based on Linux or Android

What's New in 2.3.x

New features in version 2.3.0 include:

- Support for U.are.U 5200 and 5300 fingerprint readers
- TouchChip Device Add-On - An optional component providing capture-only support for the Eikon family of fingerprint readers including the Eikon Touch 510/710 and the Eikon II. For additional information, see the readme.txt file included with the add-on.

How the U.are.U Manuals are Organized

The U.are.U documentation set consists of a family of books:

- The U.are.U SDK Developer's Guide (this manual) describes the architecture and organization of the SDK plus an overview of language/framework support.
- Platform Guides for Linux, Windows, Windows CE and Android provide details about installation, requirements and code samples for devices based on Linux, Windows, Windows CE and Android operating systems, respectively.

In addition, we provide Javadoc and Doxygen documentation that provides the detail of method parameters and data structures for the .NET, ActiveX and Java APIs.

Getting Updated Documentation

If you are viewing this developer guide from the download package for the U.are.U SDK, you may want to check our website for the latest version of this document.

http://www.crossmatch.com/Support/Reference-Material/SDK-Reference-Material/

Target Audience

This manual is aimed at developers who already have a working knowledge of their development environment and their chosen reader platform. We make the following assumptions.

- C/C++ API - We assume you have a working knowledge of the C++ language.
- .NET - We assume that you know how to develop for .NET and understand .NET concepts like static classes, IDisposable and IEnumerable.
- ActiveX - We assume that you know how to develop with ActiveX.
- Java - We assume a working knowledge of the Java language.
- OPOS and JavaPOS - We assume knowledge of the UPOS (OPOS and JavaPOS) specification version 1.13 and familiarity with Point of Sale applications and developing with OPOS/JavaPOS.

How this book is arranged

Introduction (this chapter), provides an overview of the features and standards compliance of the U.are.U SDK.

Background, contains a brief introduction to fingerprint recognition and fingerprint biometrics.

Developing Applications, describes the context and issues for developing applications using the U.are.U SDK. This chapter shows how data flows among the U.are.U SDK components, and typical workflows.

Working with Fingerprint Readers contains information about working with fingerprint reader hardware.

Upgrading from previous SDKs, discusses how to upgrade your application to use the current API functions and data formats, as well as a mapping between the One Touch terminology and the terminology used in the U.are.U SDK.

Using the SDK, provides an overview of the SDK and describes how to generate API documentation using Doxygen.

The C/C++ APIs, provides an overview of the functions in the C libraries.

The JAVA API, describes the interfaces for the Java libraries.

The .NET API, describes the classes and methods in the .NET libraries as well as the .NET controls.

ActiveX, describes the interfaces for the ActiveX libraries and controls.

JavaPOS, describes the JavaPOS API including data management, implementation notes, error codes and exceptions.

OPOS, describes the OPOS API including data management, implementation notes, error codes and exceptions.

Application Notes, provides additional suggestions for getting your application to work.

Glossary, lists the terminology specific to fingerprint recognition applications and to the U.are.U SDK.

Data Formats and Standards

The DigitalPersona U.are.U SDK supports three data types:

- DigitalPersona
- ANSI
- ISO

Background

THIS CHAPTER PROVIDES BACKGROUND INFORMATION ON THE BASICS OF FINGERPRINT RECOGNITION

In this chapter, we discuss the basics of fingerprint recognition. This chapter is not intended to be exhaustive, rather we're going to give you enough background knowledge to develop your own application more effectively.

For a more detailed overview, we recommend the Handbook of Fingerprint Recognition by D. Maltoni, M. Maio, A. Jain, and S. Prabhakar, published by Springer, 2nd edition, 2009.

Biometrics

Identifying individuals based on their distinctive anatomical (fingerprint, face, iris, hand geometry) and behavioral (signature, voice) characteristics is called biometrics. Because biometric identifiers cannot be shared or misplaced, they intrinsically represent an individual's identity. Biometrics is quickly becoming an essential component of effective identification solutions. Recognition of a person by their body, then linking that body to an externally established "identity", forms a powerful authentication tool.

Biometric identification helps to reduce fraud, and enhance user convenience. Among the different biometric identification methods, fingerprint recognition technology has a good balance of qualities including accuracy, throughput, size and cost of readers, maturity of technology and convenience of use, making it the dominant biometric technology in commercial applications.

Features of Biometric Technology

Biometric solutions offer many advantages that other technologies cannot provide.

- Uniqueness - Fingerprints from each one of our ten fingers are distinctive, different from one another and from those of other persons. Even identical twins have different fingerprints.
- Convenience - Users no longer have to remember multiple, long and complex, frequently changing passwords or carry multiple keys.
- Non-repudiation - Ensures the user is present at the point and time of recognition and later cannot deny having accessed the system.
- Non-transferable - Cannot be shared, lost, stolen, copied, distributed or forgotten unlike passwords, PINs, and smart cards.
- Proven - Long history of successful use in identification tasks - the U.S. and other countries have extensive real-world experience with fingerprint recognition. Fingerprints have been used in forensics for well over a century and there is a substantial body of scientific studies and real world data supporting the distinctiveness and permanence of fingerprints.

The Basics of Fingerprint Identification

- The skin on the inside surfaces of our hands, fingers, feet, and toes is "ridged" or covered with concentric raised patterns. These ridges are called friction ridges and they provide friction making it easier for us to grasp and hold onto objects and surfaces without slippage. The many differences in the way friction ridges are patterned, broken, and forked make ridged skin areas, including fingerprints, distinctive.
- The distinctiveness of fingerprints is well established. The underlying biological persistence of fingerprint characteristics is also a well established fact reported in various fingerprint studies conducted in different scientific fields over the past century.

Issues in Fingerprint Recognition Technology

In a perfect world, it would be a simple matter to determine whether two fingerprints were from the same finger-- the images would be identical or they would not. However, even though our fingerprints do not change over time, the fingerprint images can vary a lot, especially for some people. For example, certain skin conditions and wear due to manual labor can affect fingerprint images. This makes fingerprint recognition a very challenging problem that does not have a perfect solution. As a result, captured fingerprint images are often not a perfect match to the stored image from the same finger.

Fingerprint images from two different fingers of two different people can look similar, especially when, because of worn fingerprints or temporary creases, there is very little information left about the actual fingerprint. The larger the population you are working with, the more likelihood of similar fingerprint images.

Fingerprint recognition software needs to address these issues. We'll discuss that more in later sections of this guide.

Developing Applications

THIS CHAPTER DESCRIBES THE PROCESS OF DEVELOPING FINGERPRINT RECOGNITION APPLICATIONS.

This overview covers the following topics.

- How fingerprint recognition works
- Data structures and data flows among components
- Typical workflows
- Design issues and tradeoffs.

The JavaPOS and OPOS APIs conform to the UPOS specification and therefore do not have exactly the same data structures and workflow as the other APIs in the U.are.U SDK. See the following chapters, “JavaPOS” and “OPOS” for details.

How Fingerprint Recognition Works

Fingerprint recognition works in two stages:

- First, users are enrolled with the system--their fingerprints are captured and stored in a database.
- Next, when a person needs to be given access (e.g., to open a door or to log in to a computer), they simply scan their finger on the fingerprint reader.

In terms of application development, this typically requires the developer to build the following components:

- An application that enrolls users:
  - Captures multiple fingerprints for at least two fingers from a fingerprint reader.
  - Checks image quality to ensure that a good quality scan is obtained.
  - Extracts the fingerprint minutiae.
  - Saves the fingerprint images and/or minutiae in a database.
- A service(s)/application(s) that identifies/verifies people:
  - Captures a fingerprint from a fingerprint reader.
  - Extracts the fingerprint minutiae.
  - Compares fingerprint with enrolled fingerprints to identify a user from a list or verify a specific user.

This SDK provides fingerprint capture, extraction, enrollment and identification/verification functions to help you develop these components.

Understanding the Data Flow

When building a fingerprint recognition application, the data flow consists of:

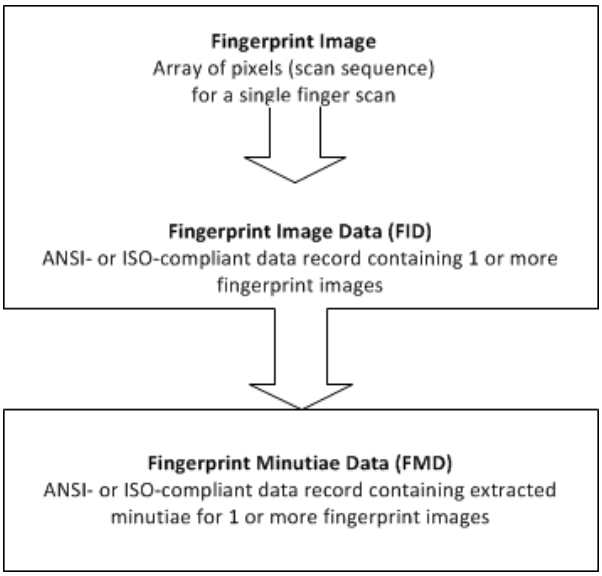
1 Capture a Fingerprint Image (scan) from the fingerprint reader. The resulting Fingerprint Image Data (FID) contains one or more fingerprint images, called a Fingerprint Image Views (FIVs). A typical FID for fingerprint recognition applications contains only one FIV but we also support multiple views (e.g., if there are multiple fingers from one individual or multiple images from a single finger stored in a single FID).

   Each FIV (fingerprint) is approximately 140K in size.

2 Extract the fingerprint features. During extraction, Fingerprint Minutiae Data (FMD) is created, with each fingerprint stored in a Fingerprint Minutiae View (FMV) in the FMD. An FMV in an FMD takes no more than 1.5K (maybe less depending on the fingerprint). FMDs are used for identifying users in a collection and verifying specific users.

The ANSI and ISO standards permit multiple views but the U.are.U SDK creates only single-view FIDs and FMDs.

This data flow is shown in Figure 1 below.



A terminology note: in the past, enrollment fingerprints were stored as templates and fingerprints to be identified/verified were created as feature sets. This data model is still supported when using the DigitalPersona data format - that is, there are two kinds of FMD, depending on whether the data is for an enrolled fingerprint or for a fingerprint to be identified/verified. When using ANSI and ISO standard data, the template/feature set terminology does not apply since the ANSI/ISO standards do not make that distinction. The data flow for legacy SDK data may be different, as described in Working with Data Created with Gold and One Touch SDKs on page 19.

Workflow - Enrollment Application

During the enrollment process, one or more fingers are scanned for each person. We recommend that you enroll at least two fingers (more is recommended) because in the event of an accident or injury to one finger, another enrolled finger can be used to identify the individual.

The enrollment application needs to perform the following steps to enroll a single finger from a user:

Step One - Initialization

Initialize the library. Discover the available readers and open a connection to the reader.

Step Two - Capture, Extract and Enroll

1 Begin the enrollment process.

2 Capture a series of fingerprint scan(s); for each scan,

- Create an FID,
- Extract fingerprint minutiae and create the FMD,
- Add the FMD to the pool of FMDs for enrollment.

3 Continue to capture fingerprints until the enrollment process has enough FMDs to complete the enrollment. (The enrollment functions evaluate the FMDs and select the best image -- typically several scans are required.)

4 Create the enrollment FMD and release resources.

Step Three - Store Data

Store the enrollment FMD. Many applications keep only the enrollment FMD because of space constraints or policy decisions. You cannot use FIDs for identification, so even if you choose to keep the FIDs, you must also store the FMD for each individual.

Notes on Enrollment

Before storing, you may want to check for existing entries that match the new entry -- applications like law enforcement, banking or voting registration, may not allow duplicate enrollments.

The capture/extract minutiae part of the enrollment process is the same as for capturing/extracting minutiae for the purpose of verifying/identifying users. If you wish, you can enroll users without using the enrollment functions (by simply capturing, extracting minutiae and storing the resulting FMD). However we recommend that you use the enrollment functions to create the best quality enrollment FMDs.

The enrollment process is slightly different in each API. Consult the chapters that describe the various APIs to determine the specifics for your language. For JavaPOS and OPOS, the enrollment process is described in the specification.

Workflow - Identifying/Verifying

Fingerprint recognition involves two types of operation:

- **Identification** - Comparing a fingerprint against the database of enrolled fingerprints and confirming that the fingerprint is enrolled (e.g., to open a door there may be many authorized users).
- **Verification** - Comparing a fingerprint against a specific user's enrolled fingerprint(s) to verify a specific person's identity (e.g., when the user types their name and then uses a fingerprint rather than a password).

To perform these operations, your application needs to do the following steps:

Step One - Initialization

Initialize the library. Discover the available readers and open a connection to a reader.

Step Two - Capture and Extract

Wait for a fingerprint. When a fingerprint is detected, capture the image and create an FID.

Extract fingerprint minutiae and create an FMD.

This sequence is exactly the same as for the capture/extraction process during enrollment.

Step Three - Identify/Verify

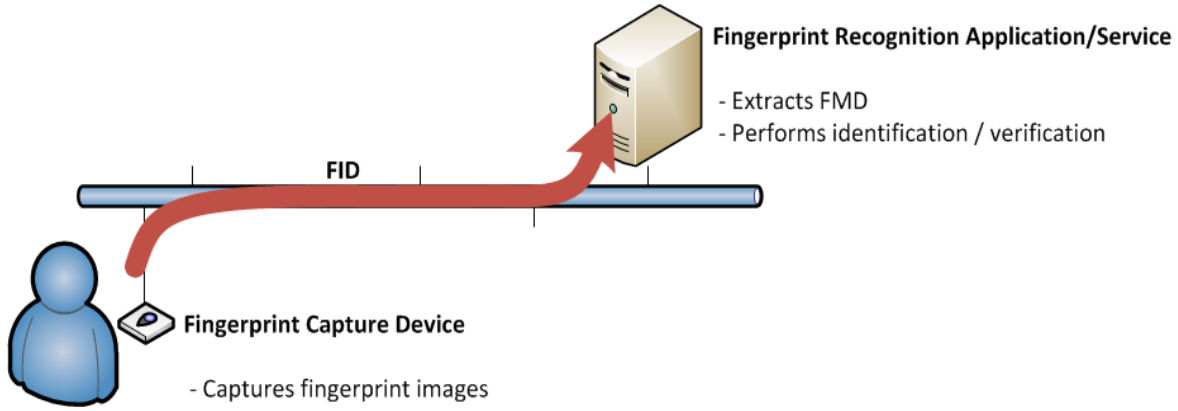
Call the appropriate function to verify a specific person OR to identify a valid user.

Design Issues for your Application

Distributed Processing and Data Flow

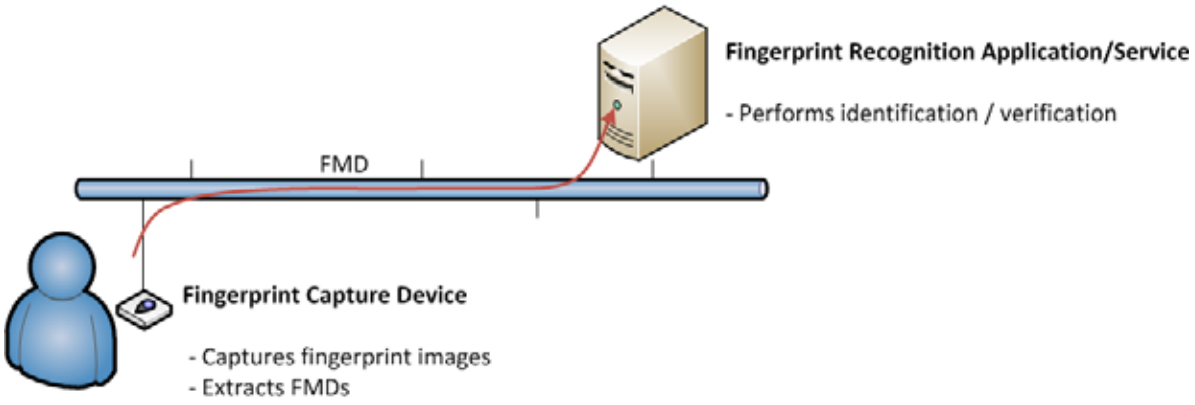
Depending on the capabilities of your fingerprint reader, you can capture FIDs and send them to another machine for processing OR the fingerprint reader can extract the FMD and transmit only the much smaller FMD files. Thus the application can be designed in these two ways:

1 The fingerprint capture device can simply capture a fingerprint image and transmit the image to a server for processing as shown in the image below. Since FIDs are large (around 100K - 140K), this means that you need a faster connection, but there is less computing power required by the fingerprint reader.



Optionally, the fingerprint capture device can compress the image before transmitting it.

2 Another alternative is to develop software for the fingerprint capture device to capture the fingerprint image AND extract the fingerprint features to create an FMD. The FMD is then transmitted to the server for processing, as shown below. FMDs are 1.5K or less and so require less bandwidth and speed.



Data Compression

If you use compression to save bandwidth or storage space, there are two options:

1 U.are.U provides compression using Wavelet Scalar Quantization (WSQ). WSQ is a wavelet-based compression standard developed by NIST specifically for fingerprint data.

2 You can compress the data using a lossless compression such as JPEG2000.

WSQ Compression

WSQ compression allows compression of 15:1 to 12:1. For more details, see Wavelet Scalar Quantization (WSQ) Compression on page 26.

JPEG2000 Compression

If you are using JPEG2000 as a compression algorithm, choose 12 as the quality setting. This allows lossless compression at around 10:1. The U.are.U SDK does not support JPEG2000 compression directly, so you would have to capture the fingerprint as a raw image and compress it to JPEG2000. After transmission, you must convert the JPEG2000 file back into a raw image for minutiae extraction and identification/verification.

NIST Fingerprint Image Quality (NFIQ)

The NFIQ score indicates the quality of a fingerprint sample. The U.are.U SDK provides NFIQ calculation using code developed by NIST. The NFIQ score is in the range 1 – 5, with 1 being the best and 5 being not suitable for feature extraction. For more details, see NIST Fingerprint Image Quality (NFIQ) on page 27.

Determining an Acceptable Level of Error

When identifying fingerprints, you want to identify strictly enough that you do not let unauthorized people have access (false positives) but also do not inconvenience legitimate users by rejecting their fingerprints (false negatives). Note that some people will always experience more false rejections -- the rate of false negatives is a statistical measure, but individuals may experience higher rejection rates, based on their specific fingerprint characteristics.

There is a trade-off between the frequencies of false positive and false negative errors. Applications have control over this trade-off by specifying the threshold for the required degree of similarity between two fingerprint images in order to call it a match.

When choosing the identification threshold, note that increasing the false positive error rate by a factor of 100 will reduce the false negative error rate only approximately by a factor of 2. There will always be some false negatives. As the result, every practical fingerprint recognition system should have an alternative means to establish and prove identity, without using fingerprints.

Setting the Error Threshold when Identifying a Fingerprint in a Collection

When a fingerprint is scanned, the first step is to identify the fingerprint against a set of stored FMDs.

- If you are trying to confirm that a user is allowed access, you will want to identify the fingerprint against all the valid FMDs that you have stored.
- If you are trying to confirm the identity of a specific person, you must identify against all FMDs for that individual (typically at least two fingers are stored for each user).

The identification function compares an FMV against a collection of FMDs to produce the candidate list. You can specify the maximum desired number of candidates: a smaller number can make the execution faster. The candidate list is sorted by the dissimilarity score, the lower the score the closer the candidate to the beginning of the list. The best match is the first candidate in the list.

Your **threshold** determines the trade-off between false positive and false negative error rates where:

- 0 = no false positives
- maxint (#7FFFFFFF or 2147483647) = fingerprints do not match at all
- Values close to 0 allow very few false positives; values closer to maxint allow very poor matches (a lot of false positives) in the candidate list. The table below shows the relationship between the threshold values and the false positive identification error rates observed in our test. Note: the actual false positive identification error rates in your deployment may vary.

Your Threshold	Corresponding False Positive Identification Rate	Expected number of False Positive Identifications	Numeric Value of Threshold
.001 * maxint	.1%	1 in 1,000	2147483
.0001 * maxint	.01%	1 in 10,000	214748
.00001 * maxint	.001%	1 in 100,000	21474
1.0e-6 * maxint	.0001%	1 in 1,000,000	2147

For many applications, a good starting point for testing is a threshold of 1 in 100,000. If you want to be conservative, then you will want to set the threshold lower than the desired error rate (e.g., if you want an error rate that does not exceed 1 in 100,000, you might set the threshold to 1 in 1,000,000).

Defining the Data Retention Policy

After a fingerprint scan, an FID is created, which contains the actual image. Each fingerprint image takes roughly 140K of storage. With modern computers, that is not a huge amount, but each enrolled user may have several fingerprints scanned. Multiplied by the number of potential users, this can add up to a fair amount of data.

To identify fingerprints, you must extract the fingerprint characteristics to create an FMD, which is < 1.5K per fingerprint.

Some applications choose to retain the full image records, but other companies discard the image record and retain only the minutiae data in the form of an FMD.

IMPORTANT: If you discard the image record, you cannot reconstruct the original fingerprint image from the FMD, the FMD is only useful for identifying/verifying fingerprints.

From time to time Crossmatch may release a new version of the DigitalPersona U.are.U SDK that will provide improved accuracy in the feature extraction process. If you do not retain the fingerprint images, you will not be able to redo the feature extraction using the new version of the SDK/runtime in order to take advantage of these improvements.

Specifying the Fingers to Be Scanned

When you enroll people into your system, you will usually want more than one finger enrolled. This allows for injury and makes it easier for people who have fingerprints that are difficult to recognize. For some applications you may want to scan all ten fingers or take multiple fingerprint impressions for individual fingers.

A typical policy would be to require both index fingers or both thumbs to be scanned. Thumbs are typically the worst in terms of recognition accuracy and thumbs require different capture devices with larger area and different ergonomics. Where possible, avoid any industrial design forcing users to use thumbs.

The preferred approach is to enroll, at a minimum, both index and both middle fingers. Middle fingers are usually the best, probably because people have almost as good dexterity with middle fingers as with index fingers, and yet middle fingers have fingerprints that are typically less worn than index fingers.

Some solutions are ergonomically designed in a way that only the right or only the left hand can be used conveniently. In this case the right hand is better (probably because the majority of people are right handed), and at least three fingers need to be enrolled: index, middle and ring fingers.

Ergonomics and the correct finger placement are extremely important. Poor ergonomics can easily increase the false negative identification rate by a factor or 5 to 10. The system users need to be aware of correct finger placement for best results.

Optimizing Fingerprint Applications

To identify a fingerprint against a large database can take a considerable amount of time and create unacceptable delays between the fingerprint scan and the user authorization. Identifying fingerprints will be faster if the database of fingerprints is in memory rather than retrieved from disk. If you have a large database of users, you may need to provision your server with an appropriate amount of RAM to handle the searches.

This SDK is not optimized for large scale identification. If you are developing such an application, you may want to contact Crossmatch to get help selecting the technology that will best fit your needs.

To ensure the fastest response time, you must weigh whether it will be faster to transmit the image record to a server for FMD extraction or whether it is faster to extract the FMD on the reader and transmit only the FMD to the server for identification/verification.

For readers that are used by a limited number of people (e.g., kiosks or pharmacy cabinets), you may have the device identify fingerprints against a limited set of FMDs. However this requires that you keep the FMDs in the device in sync with your central database, to ensure that new employees are able to gain access and departing employees' privileges are revoked quickly.

Working with Fingerprint Readers

THIS CHAPTER PROVIDES INFORMATION ABOUT SUPPORTED FINGERPRINT READERS AND THEIR USE.

The DigitalPersona U.are.U®SDK works with the following fingerprint readers:

- U.are.U 4000B Fingerprint Reader Rev. 100 (all platforms) and Rev. 101 48Mhz (Windows CE only)
- U.are.U 4500 Fingerprint Reader Rev. 103
- U.are.U 5xxx Fingerprint Readers (5100, 5160, 5200 and 5300)

However, please note the following limitations.

- The 4xxx series readers have limited support for streaming.
- Only two 5xxx readers may be used simultaneously, and they must be connected to separate USB controllers.
- Not all platforms have complete support for all readers -- check the platform guides for more details.

Fingerprint readers can lose their calibration as a result of changes in temperature, humidity and ambient light. Humidity is the most important environmental factor affecting calibration. U.are.U fingerprint readers are self-calibrating, but you still might want to set up your application to check periodically that no additional calibration is needed.

If the fingerprint reader is not giving clear readings:

- Try cleaning it:
  - For the 4xxx readers, clean the gel surface with sticky tape. Gently dab it with the sticky side of the tape. Do not rub it with paper and do not get it wet.
  - For the 5xxx readers, in addition to sticky tape, you can use a damp wipe. Do not clean with compressed air and do not use industrial cleaners or solvents.
- Make sure that you are touching the fingerprint reader with the pad of your finger, not the tip. The most detail (minutiae) occur roughly midway between the first joint and the tip.
- If your fingers are very dry, try touching your forehead with the pad of the finger you are trying to scan and then rescanning your fingerprint.

Good ergonomics in the mounting of your fingerprint reader can significantly improve the quality of fingerprint scans. Be sure that the reader is mounted in a way that is convenient for users to touch properly, such that the large area of the pad of the finger is captured. The angle at which the reader is mounted as well as the rim around the sensing area can make it difficult for people with large fingers or long fingernails to have proper contact between the pad of the finger and the sensing area.

If your fingerprint reader becomes non-responsive from an electrostatic shock you may need to perform a hardware reset.

Your application should check the reader status between fingerprint scans to ensure that the hardware has not experienced an error condition.

Upgrading from previous SDKs

THIS CHAPTER DESCRIBES THE PROCESS OF DEVELOPING FINGERPRINT RECOGNITION APPLICATIONS.

Overview of Support for Previous SDKs

Working with Existing Data Created with Previous SDKs

The DigitalPersona U.are.U®SDK supports DigitalPersona, ANSI, and ISO formats. You may choose your format according to your application requirements.

However, the data formats are not inter-operable and only one data format can be used in a single application.

For example, when passing an array of fingerprint templates into the identification function, all the templates in the array must be in the same format. If your application has existing data in a specific format, you must continue to use that format for all of your data.

IMPORTANT: *It is not possible to convert data from one format to another. If you decide to move from one format to another, you must re-enroll your user population.*

Exchanging Data with Applications Using Previous SDKs

Applications written with the U.are.U SDK APIs can exchange data with applications developed using previous DigitalPersona products. The table below shows which U.are.U APIs may be used to develop applications that will exchange data with existing applications:

Previous DigitalPersona SDK used by existing application	Compatible U.are.U APIs
Gold SDK/Fingerprint Recognition Software	C/C++, Java, ActiveX
One Touch SDKs	C/C++, Java, ActiveX
U.are.U UPOS for JavaPOS	JavaPOS
U.are.U UPOS for OPOS	OPOS

The U.are.U SDK does not support the custom encryption keys that are supported by the Gold and One Touch SDKs.

Upgrading Applications from Previous SDKs

To convert applications developed using Gold and One Touch SDKs to use the U.are.U SDK requires software modifications to use the U.are.U APIs. The U.are.U APIs are described in detail in the other chapters of this manual.

Applications developed with U.are.U UPOS for JavaPOS or UPOS for OPOS can be upgraded to use the JavaPOS and OPOS APIs of the U.are.U SDK, since both the previous and the current product support the UPOS specification. More details are provided below in Upgrading from U.are.U UPOS for OPOS/JavaPOS on page 20.

Converting Applications from the One Touch SDK

To convert an application from the One Touch SDK, install the U.are.U SDK as described above and use this documentation to modify your applications to use the new API.

Be sure to install the U.are.U SDK in a new folder on your development machine so that your existing files do not get overwritten.

Note that when you install the U.are.U SDK on the target machine, the One Touch drivers will be overwritten with new drivers that are compatible with both your existing applications and new applications based on the U.are.U SDK.

The files installed on the target machine include both drivers and SDK files. If you retain the One Touch SDK files on the device, you will need up to an additional 120K for the new U.are.U SDK files. If you retain the old SDK files on

the device, you can run applications based on either the old or the new SDK. The two SDKs can coexist and your existing applications can run using the older run-time environment while you update your programs or develop new applications based on the new SDK. Applications based on the old and new SDKs can run on the same hardware, but not simultaneously.

Note that the ***data*** from existing applications based on the One Touch SDK is fully compatible with applications based on the U.are.U SDK.

Working with Data Created with Gold and One Touch SDKs

The DigitalPersona Gold SDK and One Touch SDK products used a different format for minutiae data (which were called feature sets). In the Gold and One Touch formats, there were three different data types for minutiae data which reflected the intended use of the data:

- ***Pre-registration features (***Gold SDK), or ***Pre-registration Feature Set*** to be used for Enrollment (OneTouch SDK); Pre-registration features were intended only for creation of Registration features. An application needed to collect four fingerprints of the finger to enroll, extract pre-registration features, and pass it to the SDK to produce registration features.
- ***Registration features*** (Gold SDK), or ***Fingerprint Template*** (OneTouch SDK); Registration features were intended to be stored in a database as an enrolled finger.
- ***Verification features*** (Gold SDK), or ***Feature Set*** (One Touch SDK). Verification features are what is compared to the Registration features when a user swipes a finger.

In these previous SDKs, you could not compare two Feature Sets, you could only compare a Feature Set against a Fingerprint Template. The U.are.U SDK removes the distinction between feature sets and templates when using the ANSI/ISO formats. For data created in ANSI/ISO data formats, all minutiae data is stored in an FMD, whether produced by the feature extraction functions or the enrollment functions.

Changes in U.are.U Terminology from Previous Usage (Gold and One Touch)

This section describes changes in terminology from the One Touch documentation and other DigitalPersona products.

Old Term	New Term	Explanation
Fingerprint authentication	Fingerprint verification	Change in industry standard terminology.
Fingerprint registration	Fingerprint enrollment	Change in industry standard terminology.
Fingerprint sensor (referring to a fingerprint capture device)	Fingerprint reader	Erroneous usage: Sensors are a *component* of certain types of *fingerprint capture devices*, they are not themselves *fingerprint capture devices*.
Match Matching Matching score	Compare Comparison Comparison score	Change in industry standard terminology.
Performance	Recognition accuracy	More accurate terminology.
Fingerprint feature set Fingerprint template	Fingerprint minutiae data	*Fingerprint templates* (*fingerprint features* stored for enrolled fingers) and *fingerprint feature sets* (images used for verification and identification) have been replaced with *fingerprint minutiae data* in the U.are.U family of the SDKs only. With standards-based data formats in the U.are.U SDKs, all *fingerprints* are stored in the same format.
Fingerprint feature set Fingerprint template	Fingerprint minutiae data	*Fingerprint templates* (*fingerprint features* stored for enrolled fingers) and *fingerprint feature sets* (images used for verification and identification) have been replaced with *fingerprint minutiae data* in the U.are.U family of the SDKs only. With standards-based data formats in the U.are.U SDKs, all *fingerprints* are stored in the same format.
ROC curve	DET curve	More accurate terminology. We have never used ROC curves, however our DET curves were sometimes erroneously called ROC curves in the past.
FAR	FMR or FNMR as appropriate	The definition of *FAR* is application-specific. In some applications *FAR* is similar to *FMR* while in other applications, *FAR* is similar to *FNMR*.
FRR	FMR or FNMR as appropriate	The definition of *FRR* is application-specific. In some applications *FRR* is similar to *FNMR*, while in the others *FRR* is similar to *FMR*.

Upgrading from U.are.U UPOS for OPOS/JavaPOS

This new U.are.U SDK version of the OPOS and JavaPOS APIs are backward-compatible. However the following new features may or may not affect your existing applications:

- **The default data format has a longer header.** The default data format is now fully compliant with UnifiedPOS 1.13. The default data format is the same as the old format except that the template header has additional bytes of information, i.e., a 45-byte header, instead of the previous 10-byte header for JavaPOS and 12-byte header for

OPOS. If you do not update your application to specify a specific format, new templates will be created with 45-byte headers. Enrollment, identification and verification will continue to work with both new templates with 45-byte headers and previous templates that have the shorter headers. See Working with Fingerprint Data in JavaPOS on page 52 and Working with Fingerprint Data in OPOS on page 59 for more information.

- **New FAR Security Setting. W**e have tightened the FAR security setting to align with our current corporate standard. The new standard is 100 times more strict. Please update your code accordingly. We recommend that you allow the FAR setting to be configurable at run-time within your application. See Determining an Acceptable Level of Error on page 15 for more information on setting FAR.
- **Enrollment no longer stops at four retries.** The previous enrollment process would fail after four unsuccessful scans. If your application is dependent on having exactly four tries, then you may need to adjust your code.
- **Errors and exceptions may be handled differently.** Because of the extensive architectural changes to this version of the OPOS and JavaPOS APIs, exceptions and error codes may not be identical. You should double check carefully that the new API handles errors and exceptions as expected for your application.

For specific details about upgrading your existing application, consult JavaPOS on page 52 or OPOS on page 59.

Using the SDK

THIS CHAPTER DESCRIBES CONTENTS OF THE SDK AND THE PURPOSE OF EACH COMPONENT.

What's In the SDK?

The SDK consists of:

1 C/C++ API -- C libraries that conform to ANSI.C99 (http://en.wikipedia.org/wiki/C99):
   - **DP Capture API** - for capturing fingerprints
   - **FingerJet Engine API** - for extracting fingerprint characteristics and identifying/verifying fingerprints

2 .NET API -- .NET class libraries
   - **DP .NET API** - for capturing and comparing fingerprints
   - **DP .NET Controls** - simple interface for enrollment and identification, based on OneTouch interface
   - **DP ActiveX Library** - for capturing and comparing fingerprints using the ActiveX wrapper with .NET
   - **DP ActiveX Controls** - simple interface for enrollment and identification, based on OneTouch interface

3 ActiveX API and controls -- ActiveX class libraries
   - `DPXUru.dll` – ActiveX API library
   - `DPCtlXUru.dll` – ActiveX GUI controls

4 Java API -- Java class libraries
   - `dpuaru.jar` - library classes and interfaces for working with readers and the FingerJet Engine

5 JavaPOS API -- class libraries that implement a JavaPOS-compliant API (per the JavaPOS 1.13 specification), as a wrapper to the U.are.U Java API:
   - **`dpjavapos.jar`** - library classes and interfaces for working with readers
   - JavaPOS Device Service object, which can be used with any JavaPOS Device Control for the Biometrics device category

6 OPOS API -- classes that implement an OPOS-compliant API (per the UPOS 1.13 specification), as a wrapper to the U.are.U C/C++ API:
   - `dpServiceObject.dll` – a custom implementation of the OPOS data service
   - **OPOSBiometrics** – a custom implementation of the OPOS biometrics control

7 Run-time components:
   - Capture driver and SDK layer
   - FingerJet Engine run-time

FingerJet Engine

The FingerJet Engine is a module that extracts fingerprint characteristics from image records to create FMDs and compares FMDs to confirm identity. The FingerJet Engine has met the PIV performance thresholds for fingerprint minutiae data generation required by NIST.

We support a maximum of 16 views in a single FID or FMD during authentication. All views in a single record must have the same resolution. We do not support unknown finger positions (finger position = 0).

The C/C++ APIs

THIS CHAPTER PROVIDES AN OVERVIEW OF THE C/C++ APIS AVAILABLE FOR THE LINUX, WINDOWS AND WINDOWS CE PLATFORMS. FOR DETAILS ON USING THE API WITH A SPECIFIC OS PLATFORM, CONSULT THE APPROPRIATE PLATFORM GUIDE.

Detailed documentation for the API is contained in the header files. You can simply read the header files or you can view the Doxygen files. Consult the platform guide for details of where the Doxygen files are located.

The API is thread-safe.

DP Capture API

The DP Capture API consists of library management, reader management, capturing and streaming.

Library Management

Function	Description
dpfpdd_init	Initialize the library (allocate system resources and initialize data). **This must be the first function called.**
dpfpdd_exit	Release the library and its resources.
dpfpdd_version	Query the library version. **This is the only function that can be called before dpfpdd_init or after dpfpdd_exit.** This returns the DP Capture API library version (not the U.are.U SDK version). This is analogous to the dpfj_version function which returns the version of the FingerJet library file.

Fingerprint Capture Device Management

Function	Description
dpfpdd_query_devices	Discover connected devices.
dpfpdd_open	Open a device. This function establishes an exclusive link to the device; no other processes will be able to use the device until you close it. The application *must* open the device before use.
dpfpdd_close	Close a device.
dpfpdd_get_device_status	Get the status for a device. You would normally check the device status between captures to ensure that the device is functioning and there are no error conditions.
dpfpdd_get_device_capabilities	Query a device for information on capabilities.
dpfpdd_set_parameter	Change a device or driver parameter
dpfpdd_get_parameter	Query a device or driver parameter
dpfpdd_calibrate	Calibrate a reader. Some readers are self-calibrating. Ambient light or temperature can affect calibration, for some readers. Calibration can take several seconds.
dpfpdd_reset	Do a hardware reset on the reader. Hardware resets are typically needed only after a hardware problem (e.g., the device is unplugged or receives an electrostatic shock). Hardware resets typically only take a few milli-seconds.

Capturing Fingerprints

The `dpfpdd_capture` function captures a fingerprint image for

- Enrollment (as part of the process described on page *25)*
- Identifying users with `dpfj_identify`
- Verifying a specific user identity with `dpfj_compare`

Function	Description
dpfpdd_capture	Capture a fingerprint image from the reader. This function signals the reader that a fingerprint is expected, and waits until a fingerprint is received.
dpfpdd_cancel	Cancel a pending capture

Streaming Fingerprints

Not all readers support streaming mode. To determine if a specific reader supports this feature, check the value of `can_stream_image` in `DPFPDD_DEV_CAPS`, as returned by `dpfpdd_get_device_capabilities()`.

The streaming methods are:

Function	Description
dpfpdd_start_stream	Start streaming mode
dpfpdd_get_stream_image	Capture a fingerprint image from the streaming data
dpfpdd_stop_stream	End streaming mode

FingerJet Engine API

The FingerJet API contains functions that extract features from FIDs to create FMDs, identify/verify FMDs and convert FMDs to different formats.

Library Management

Function	Description
dpfj_version	Query the library version. This returns the FingerJet library version (not the U.are.U SDK version). This is analogous to the dpfpfdd_version function which returns the version of the DP Capture API library file.

Select Fingerprint Matching Engine

Function	Description
dpfj_select_engine	The DigitalPersona FingerJet is the default engine used if this function is not called. The FingerJet engine is available on all platforms and does not require open reader (parameter hdev can be NULL). Not every other engine is available on every platform. Some engines require a valid handle from opened reader to be supplied. Currently, you can choose from two types of FingerJet engine, versions 6 and 7. On some platforms the Innovatrics engine is also available. If this function is not called and no choice is made by the application, the default matching engine used is FingerJet version 6. See also Minex Certification on page 68.

Extract FMD

Function	Description
dpfj_create_fmd_from_raw	Creates FMD from raw image
dpfj_create_fmd_from_fid	Creates FMD from ANSI, ISO or DigitalPersona legacy format FID

Identify Fingerprint

This function is described in detail on pages *13-14*.

Function	Description
dpfj_identify	Identify an FMD: given an array of FMDs, this function returns an array of candidates that match the original fingerprint (an FMV within an FMD) within the threshold of error. Supported formats are: Gold SDK, One Touch SDK, ANSI and ISO.

Enrollment

The enrollment functions allow you to enroll a finger to create an FMD that you can store in your database. For ANSI/ISO formats, the enrollment functions create FMDs. For the legacy DigitalPersona format, the enrollment functions create a fingerprint template.

The typical process would be:

1 Call `dpfj_start_enrollment`.

2 Capture a fingerprint scan and extract an FMD, using the standard functions (`dpfpdd_capture` to capture and `dpfj_create_fmd_from_fid` or `dpfj_create_fmd_from_raw` to extract).

3 Call `dpfj_add_to_enrollment` to add the fingerprint to the potential pool.

4 Repeat the previous two steps until `dpfj_add_to_enrollment` returns a flag indicating the pool of FMDs is now sufficient to create an enrollment FMD.

5 Create the enrollment FMD with `dpfj_create_enrollment_fmd` and release resources by calling `dpfj_finish_enrollment`.

6 Store the enrollment FMD in your database. Some applications like voting, banking and law enforcement require that you check for duplicate fingerprints before storing a new fingerprint in the database.

Function	Description
dpfj_start_enrollment	Begin the enrollment process and allocate resources.
dpfj_add_to_enrollment	Add the FMD to the pool of FMDs for enrollment and return a flag indicating that the enrollment is ready (enough FMDs have been received to create the enrollment FMD)
dpfj_create_enrollment_fmd	Create FMD for enrolled finger
dpfj_finish_enrollment	Release resources used during enrollment process

Format Conversion

Function	Description
dpfj_fmd_convert	Convert FMDs from ANSI to ISO format and vice versa.
dpfj_dp_fid_convert	Convert legacy DigitalPersona image (Gold SDK and One Touch SDK) to ANSI or ISO images

Advanced Diagnostics

The majority of applications should use the `dpfj_identify` function to implement both identification and verification. However, in a few special cases, e.g., using multi-modal biometrics, or doing statistical risk assessment, the `dpfj_compare` function allows you to compare two FMVs to determine their actual degree of dissimilarity. This is useful for accuracy testing and diagnostics and is not intended to be used in final applications for actual fingerprint recognition. The `dpfj_compare` function returns a ***dissimilarity score*** with values:

- 0 = fingerprints are NOT dissimilar (i.e., they MATCH perfectly).
- `maxint` (#7FFFFFFF or 2147483647) = fingerprints are completely dissimilar (i.e., DO NOT match).
- Values close to 0 indicate very close matches, values closer to `maxint` indicate very poor matches.

The table below shows the relationship between the scores returned from `dpfj_compare` and the false match error rates observed in our test. The dissimilarity score distribution is estimated based on our internal testing, and may not be representative of the actual rate that will be observed in deployment.

Dissimilarity Score	False Match Rate
2147483	.1%
214748	.01%
21474	.001%
2147	.0001%

Function	Description
dpfj_compare	Compare two FMDs; supported formats are: Gold SDK, One Touch SDK, ANSI and ISO.

Wavelet Scalar Quantization (WSQ) Compression

The U.are.U SDK provides compression of fingerprint images with the WSQ algorithm. This compression is available in two ways:

1 Using code developed by NIST. This code is included within the U.are.U C/C++ API.

2 Using the WSQ1000 SDK from Aware, Inc. on the target system. If the WSQ 1000 SDK is installed, the U.are.U functions will call the WSQ 1000 functions to do compression and decompression instead of the native NIST-based code.
Crossmatch does not redistribute the Aware WSQ1000 SDK, it must be acquired and installed separately.

The typical sequence to compress an image would be:

- `dpfj_start_compression`
- `dpfj_set_wsq_bitrate` or `dpfj_set_wsq_size` - Set the desired size for the compressed image
- `dpfj_compress_fid` or `dpfj_compress_raw` - Compress the FID or raw image
- `dpfj_get_processed_data` - Retrieve the compressed image
n `dpfj_finish_compression`

The typical process to decompress an image would be:

- `dpfj_start_compression`
- `dpfj_expand_fid` or `dpfj_expand_raw` - Decompress an FID or raw image
- `dpfj_get_processed_data` - Retrieve the expanded image
- `dpfj_finish_compression`

Function	Description
dpfj_start_compression	Initiates WSQ compression and allocates resources.

dpfj_finish_compression	Releases resources.
dpfj_set_wsq_bitrate dpfj_set_wsq_size	These two specify the same thing: the size of the resulting compressed image, and can be used interchangeably. Setting the bitrate at 0.75 bpp to 0.9 bpp allows compression of 15:1 to 12:1. The parameter `tolerance_aw` sets the tolerance as required by the Aware WSQ1000 SDK and it is ignored when using NIST algorithm.
dpfj_compress_fid dpfj_compress_raw	Compress an FID or raw image, according to the requested size.
dpfj_get_processed_data	Retrieve the image that was just compressed/expanded.
dpfj_expand_fid dpfj_expand_raw	Expand a previously compressed FID or raw image.

NIST Fingerprint Image Quality (NFIQ)

The U.are.U SDK calculates NFIQ scores for fingerprint images. This calculation is available in two ways:

1 Using code developed by NIST. This code is included within the U.are.U C/C++ API.

2 Using the WSQ1000 SDK from Aware, Inc. on the target system. If the WSQ 1000 SDK is installed, the U.are.U functions will call the WSQ 1000 functions to do NFIQ calculation instead of the native NIST-based code.

Crossmatch does not redistribute the Aware WSQ1000 SDK, it must be acquired and installed separately.

Function	Description
dpfj_quality_nfiq_from_fid dpfj_quality_nfiq_from_raw	Calculate the NFIQ score of an FID or raw image. NFIQ scores range from 1 to 5 with 1 being the best quality and 5 indicating that the image is not suitable for feature extraction.

The JAVA API

THIS CHAPTER DESCRIBES HOW TO USE THE JAVA API.

The Java API is built as a wrapper to the C/C++ API. The Java API is available for Linux and Windows. This chapter provides an overview of the API. For details of using the API on a specific reader platform, consult the appropriate Platform Guide.

The Java API is considerably simpler to use than the C/C++ APIs and therefore generally results in:

- Easier data management
- Easier enrollment
- Faster development

Importing the U.are.U Java package

The U.are.U Java library classes and interfaces are aggregated into `dpuaru.jar`. To use the U.are.U Java library functionality import the `com.digitalpersona.uareu.*` package, and make sure to include `dpuareu.jar` into your classpath.

Getting Detailed Documentation

This chapter provides an overview of the main methods in the Java API. For a complete description of method parameters, the Javadoc documentation for the Java libraries is provided. Consult the platform guide for details of where the Javadoc files are located.

Using the Package

Main Access Point

The main access point to the U.are.U Java library is the `UareUGlobal` class. This is a static class, which allows you to acquire references to the classes related to fingerprint readers and to the FingerJet Engine:

- **To acquire a reference to ReaderCollection** use the `GetReaderCollection()` method. To destroy `ReaderCollection`, (release all system resources associated with readers and make readers available for other processes) use the `DestroyReaderCollection()` method.
- **To acquire a reference(s) to individual readers**, use the `ReaderCollection` object, which is a collection of objects of type `Reader`.
- **To acquire a reference to the FingerJet Engine** use the `GetEngine()` method. The engine does not use or allocate any system resources except memory and does not have to be destroyed explicitly.

UareUException

The `UareUException` interface describes exceptions specific to the U.are.U SDK.

Getting a List of Available Readers

The `ReaderCollection` interface provides a list of the readers connected to the machine. A list of available readers can be acquired any time with `GetReaders()` method.

Working with Readers

Each attached reader is represented with a `Reader` object. The `Reader` interface allows:

- Querying reader description and capabilities,
- Acquiring status of the reader,
- Capturing fingerprints,
- Starting and stopping video stream, and
- Resetting and calibrating the reader.

The main methods are:

Function	Description
GetDescription	Get the description of a reader. The description is available at any time (even if the device is not open). This is the only method that can be called before the `open()` method. Returns an object of type `Description` holding information about the reader hardware.
Open	Open a device and return the device capabilities. This method establishes an exclusive link to the device; no other processes will be able to use the device until you close it. The application *must* open the device before use.
GetStatus	Get the status for a device. You would normally check the device status between captures to ensure that the device is functioning and there are no error conditions. Returns an object of type `ReaderStatus` which describes the current status of the reader.
GetCapabilities	Get the capabilities of a device. Returns an object of type `Capabilities` which describes what the reader can do.
Calibrate	Calibrate a device. Some devices are self-calibrating. Ambient light or temperature can affect calibration, for some devices. Calibration can take several seconds.
Reset	Do a hardware reset on the reader. Hardware resets are typically needed only after a hardware problem (e.g., the device is unplugged or receives an electrostatic shock). Hardware resets typically only take a few milliseconds.
Close	Close a reader and release the resources associated with the reader.

Capturing Fingerprints

The `Capture` function captures a fingerprint image for

- Enrollment (as part of the process described on page *12)*
- Identifying users with `Identify`
- Verifying a specific user identity with `Compare`

The primary fingerprint capture methods are:

Function	Description
Capture	Captures a fingerprint image from the open reader. This function signals the reader that a fingerprint is expected, and blocks until a fingerprint is received, capture fails or the reader times out.

CancelCapture	Cancel a pending capture

Streaming Fingerprints

Not all readers support streaming mode. To determine if a specific reader supports this feature, get the reader capabilities with the `GetCapabilities` method and check the value of the field `can_stream`.

The streaming methods are:

Function	Description
StartStreaming	Put the reader into streaming mode. In this mode, the application must call `GetStreamImage()` to acquire images from the stream.
GetStreamImage	Capture a fingerprint image from the streaming data. After this function returns, the reader remains in streaming mode. Frame selection, scoring and other image processing are not performed by this function.
StopStreaming	End streaming mode

Accessing the FingerJet Engine

The `Engine` interface provides functionality to

- Extract fingerprint features (create FMDs),
- Identify and compare FMDs, and
- Create enrollment FMDs.

See Understanding the Data Flow on page 11 for details on enrollment and comparison terminology and data flow.

Creating FMDs from images

The `CreateFmd()` method can be used in two ways:

1 Extract fingerprint minutiae from a raw image and create an FMD.

2 Extract fingerprint minutiae from an FID and create an FMD.

The following limitations are applied to the raw images and FIDs:

- 8 bits per pixel
- no padding
- square pixels (horizontal and vertical dpi are the same)

The size of the resulting FMD will vary depending on the minutiae in a specific fingerprint.

Identification and Comparison

`Identify()` identifies a single FMD against an array of FMDs. This function takes as inputs:

- A single view in an FMD
- An array of FMDs (each FMD can contain up to 16 views) to compare
- The desired number of candidates to return
- The threshold for False Positive Identification Rate (FPIR) that is permitted

Each time a view has a score lower than the threshold FPIR, that view is marked as a possible candidate. Then when all possible candidates are identified (i.e., they meet the threshold), they are ranked by their score. Finally, the function returns as many candidates as requested, based on the candidates with the lowest dissimilarity score. For a discussion

of setting the threshold as well as the statistical validity of the dissimilarity score and error rates, consult NIST Fingerprint Image Quality (NFIQ) on page 14.

`Compare()` takes two single views from two FMDs and returns a dissimilarity score indicating the quality of the match.

The majority of applications should use the `Identify` method to implement both identification and verification. However, in a few special cases, e.g., using multi-modal biometrics, or doing statistical risk assessment, the `Compare` method allows you to compare two FMVs to determine their actual degree of dissimilarity. This is useful for accuracy testing and diagnostics and is not intended to be used in final applications for actual fingerprint recognition. The `Compare` method returns a ***dissimilarity score*** with values:

- 0 = fingerprints are NOT dissimilar (i.e., they MATCH perfectly).
- `maxint` (#7FFFFFFF or 2147483647) = fingerprints are completely dissimilar (i.e., DO NOT match).
- Values close to 0 indicate very close matches, values closer to `maxint` indicate very poor matches.

The table below shows the relationship between the scores returned from `Compare` and the false match error rates observed in our test. The dissimilarity score distribution is estimated based on our internal testing, and may not be representative of the actual rate that will be observed in deployment.

Dissimilarity Score	False Match Rate
2147483	.1%
214748	.01%
21474	.001%
2147	.0001%

Function	Description
Compare	Compare two FMDs; supported formats are: Gold SDK, One Touch SDK, ANSI and ISO.
Identify	Identify an FMD: given an array of FMDs, this function returns an array of candidates that match the original fingerprint (an FMV within an FMD) within the threshold of error. Supported formats are: Gold SDK, One Touch SDK, ANSI and ISO.

Enrollment

`CreateEnrollmentFmd()` creates and returns an enrollment FMD. It takes as input a reference to an object of type `EnrollmentCallback`. The client must implement `EnrollmentCallback.GetFmd()`. This method acquires and returns an FMD to add to the enrollment. The engine calls `EnrollmentCallback.GetFmd()` as many times as needed in order to create an enrollment FMD.

Normally, the client application will implement `EnrollmentCallback.GetFmd()` to provide the onscreen UI to capture fingerprints from the reader, extract features using `CreateFmd()`, and return an enrollment FMD. If the user wants to cancel the enrollment, `EnrollmentCallback.GetFmd()` should return `null`.

The .NET API

THIS CHAPTER DESCRIBES HOW TO USE THE .NET API.

The .NET API is built as a wrapper to the C/C++ APIs. The .NET API is available for Windows and Windows CE. This chapter provides an overview of the API. For details of using the API on a specific reader platform, consult the appropriate Platform Guide.

The .NET API is considerably simpler to use than the C/C++ APIs and therefore generally results in:

- Easier data management
- Easier enrollment
- Faster development

The .NET API also implements additional features:

- Serialization
- Pre-built GUI controls for enrollment and identification to quickly get you started

Importing the U.are.U .NET package

The U.are.U .NET library classes are aggregated into the following class libraries:

- **DP .NET API** - for capturing and comparing fingerprints
- **DP .NET Controls** - simple interface for enrollment and identification, based on OneTouch interface

Getting Detailed Documentation

This chapter provides an overview of the main methods in the .NET API. For a complete description of method parameters, the Doxygen documentation for the .NET libraries is provided. Consult the platform guide for details of where the Doxygen files are located.

Using the Package

Main Access Points

- **To acquire a reference to ReaderCollection** use the `GetReaders()` method. To destroy `ReaderCollection`, (release all system resources associated with readers and make readers available for other processes) use the `Dispose()` method.
- **To acquire a reference(s) to individual readers**, use the `ReaderCollection` object, which is a collection of objects of type `Reader`.
- **To work with the FingerJet Engine, instantiate one of the classes:** `Comparison`, `Importer`, `FeatureExtraction` **or** `Enrollment`. For example, to start enrolling fingerprints using the FingerJet engine, call the static method, `Enrollment.CreateEnrollmentFmd()` . This method takes as input an enumeration that specifies the format the enrollment template should be in, e.g., ANSI or ISO. This method also takes an `IEnumerable<Fmd>` object. See IEnumerables in the .NET Wrapper on page 33 for more information on `IEnumerable<Fmd>`.

SDKException

The `SDKException` class describes exceptions specific to the U.are.U SDK.

Serialization

The .NET API provides the ability to convert an FMD or FID into a format that can be read serially. Serialization allows you to easily transmit and store data as byte strings, streams or XML. This allows you to transmit the data or save it to standard file systems. The XML format can be used within HTML for building browser-based applications.

To return the data to its original format, *deserialization* is also provided.

Note that serialized FMDs and FIDs include the version number of the .NET wrapper was used for the serialization. If you attempt to deserialize with an older version of the wrapper, the results may not be correct. So when deserializing, the .NET wrapper will throw an SDKException if the serialization was done using a later version of the wrapper.

Serialization in the .NET Wrapper occurs using the default `System.Xml.Serialization.XmlSerializer`. The name of the root element of the generated XML is the same as the object type. Each public member of the object is represented as an XML element. Raw byte data is encoded as a Base-64 string by `XmlSerializer`.

Deserialization uses the same `XmlSerializer`. The XML that was serialized is fed into `XmlSerializer` where the elements with values become object members with values of the same name as the element.

IEnumerables in the .NET Wrapper

The .NET wrapper uses the IEnumerable<T> interface (as specified here: http://msdn.microsoft.com/en-us/library/9eekhta0.aspx). An IEnumerable<T> is a data structure of type T that can be iterated through. An array is one example of IEnumerable<T>, because it can be iterated, or looped through.

An advantage of IEnumerable is that many different data structures can be used instead of requiring a very specific type of data structure, such as a 1-dimensional array of type T.

Perhaps the most important advantage of IEnumerables is that whatever is responsible for sending an IEnumerable to a method may use special semantics such as '`yield return`' to return only a single item at a time. This helps performance, and also helps programmers simplify their code by allowing .NET wrapper methods to pull only what is necessary for the operation.

For example, `Enrollment.CreateEnrollmentFmd(IEnumerable<Fmd>)` can iterate through a user's FMDs until enough data is captured and then create an enrollment FMD, as opposed to having more data than is necessary. Each iteration can cause a capture workflow to occur which causes '`yield return`' to return a single item to Enrollment.

NOTE: The `yield return` feature of IEnumerable is available in C# but **NOT** in VB.NET.

For more information on IEnumerable<T>, consult the Microsoft documentation pages listed below.

Topic	Microsoft Documentation
IEnumerable<T>	http://msdn.microsoft.com/en-us/library/9eekhta0.aspx
Yield	http://msdn.microsoft.com/en-us/library/9k7k7cf0(v=vs.80).aspx

Working with Readers

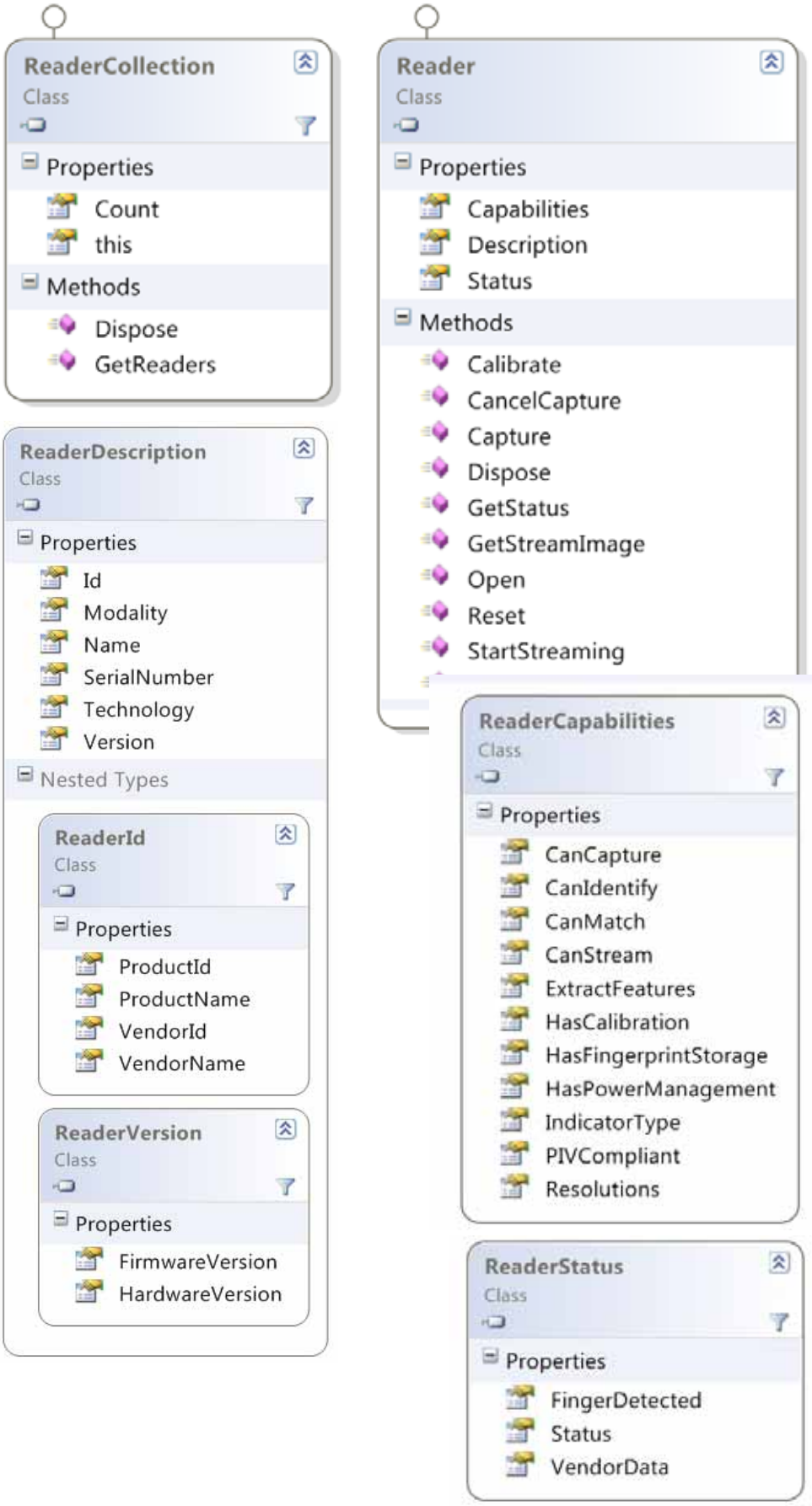
Readers are accessed through the `ReaderCollection` object, which is of type `IEnumerable<Reader>`.

Each attached reader is represented with a `Reader` object. The `Reader` interface allows:

- Querying reader description and capabilities,
- Acquiring status of the reader,
- Capturing fingerprints,
- Starting and stopping image stream, and
- Resetting and calibrating the reader.

To acquire a reader, call `ReaderCollection.GetReaders()` which returns the attached readers. Iterate through this list to look at the description object for the desired reader. Once finished with all of the `Reader` objects in the `ReaderCollection`, ensure that you call the `Dispose()` method to un-instantiate the `ReaderCollection` object and release system resources.

The UML diagrams below show the `ReaderCollection` and `Reader` classes.



The main methods for managing reader hardware in the `Reader` class are:

Function	Description
Open	Open a device and return the device capabilities. This method establishes an exclusive link to the device; no other processes will be able to use the device until you close it. The application *must* open the device before use.
GetStatus	Get the status for a device. You would normally check the device status between captures to ensure that the device is functioning and there are no error conditions. Returns an object of type `ReaderStatus` which describes the current status of the reader.
Calibrate	Calibrate a device. Some devices are self-calibrating. Ambient light or temperature can affect calibration, for some devices. Calibration can take several seconds.
Reset	Do a hardware reset on the reader. Hardware resets are typically needed only after a hardware problem (e.g., the device is unplugged or receives an electrostatic shock). Hardware resets typically only take a few milliseconds.
Dispose	Close device, release memory, remove child objects.

Capturing Fingerprints

The `Capture` function captures a fingerprint image for

- Enrollment (as part of the process described on page *12)*
- Identifying users with `Identify`
- Verifying a specific user identity with `Compare`

The primary fingerprint capture methods are:

Function	Description
Capture	Captures a fingerprint image from the open reader. This function signals the reader that a fingerprint is expected, and blocks until a fingerprint is received, capture fails or the reader times out.
CancelCapture	Cancel a pending capture

Streaming Fingerprints

Not all readers support streaming mode. To determine if a specific reader supports this feature, check the value of the reader property `CanStream`.

The streaming methods are:

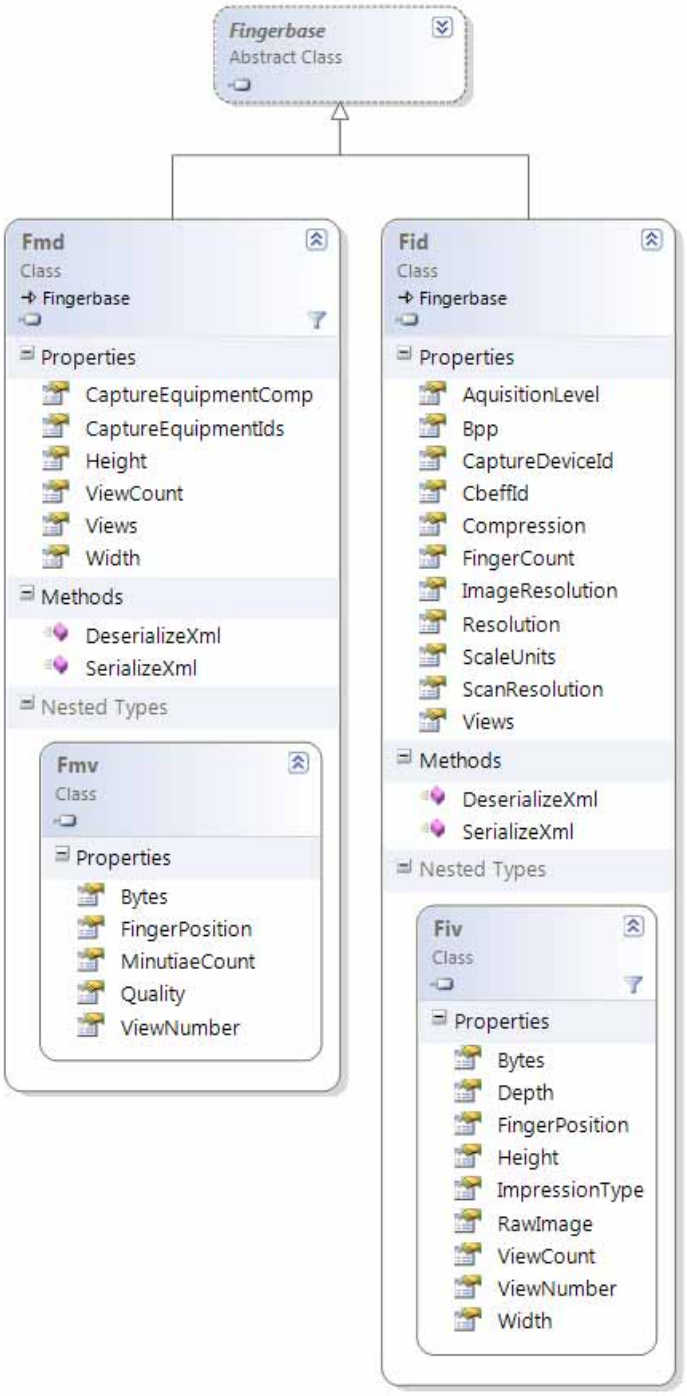
Function	Description
StartStreaming	Put the reader into streaming mode. In this mode, the application must call `GetStreamImage()` to acquire images from the stream.
GetStreamImage	Capture a fingerprint image from the streaming data. After this function returns, the reader remains in streaming mode. Frame selection, scoring and other image processing are not performed by this function.
StopStreaming	End streaming mode

Managing Fingerprint Data

The .NET API implements fingerprint data as shown in the UML diagram below and on the next page.

In addition to the usual methods for working with data, this class includes the methods below for serializing and deserializing (as described in Importing the U.are.U .NET package on page 32.

Function	Description
SerializeXml	Static method which serializes FMDs and FIDs into an XML string. Note that raw data is encoded as base-64.
DeserializeXml	Static method which de-serializes XML-encoded FMDs and FIDs back into a U.are.U FMD or FID data structure.



Analyzing and Managing Fingerprints (FingerJet Engine)

The FingerJet Engine interfaces provide functionality to

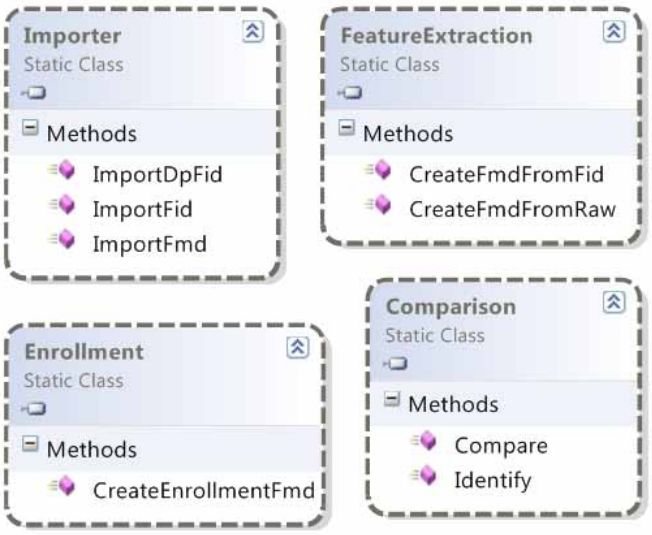
- Extract fingerprint features (create FMDs),
- Identify and compare FMDs, and
- Create enrollment FMDs.

Refer to Understanding the Data Flow on page 11 for a description of enrollment and comparison terminology and data flow.

The Classes for working with data (including identify and verify) are:

Function	Description
Comparison	This class allows you to perform: • Verification • Identification
Enrollment	To enroll a user, you must call: `CreateEnrollmentFMD` Enrollment is implemented as a static class -- you can provide an `IEnumerable<Fmd>` or a function that returns such.
Importer	The DigitalPersona U.are.U®SDK supports multiple formats for FMDs and FIDs. You can import any supported format.
FeatureExtraction	Extract features from an FID or raw image.

The UML diagram below provides more detail.



Creating FMDs from images

The `CreateFmdFromFid()` method can be used in two ways:

1. Extract fingerprint minutiae from a raw image and create an FMD.
2. Extract fingerprint minutiae from an FID and create an FMD.

The following limitations are applied to the raw images and FIDs:

- 8 bits per pixel
- no padding
- square pixels (horizontal and vertical dpi are the same)

The size of the resulting FMD will vary depending on the minutiae in a specific fingerprint.

Comparing Fingerprints

`Identify()` identifies a single FMD against an array of FMDs. This function takes as inputs:

- A single view in an FMD
- An array of FMDs (each FMD can contain up to 16 views) to compare
- The desired number of candidates to return
- The threshold for False Positive Identification Rate (FPIR) that is permitted

and returns matches as an array of integer pairs which provide the finger index and view index of each match.

Each time a view has a score lower than the threshold FPIR, that view is marked as a possible candidate. Then when all possible candidates are identified (i.e., they meet the threshold), they are ranked by their score. Finally, the function returns as many candidates as requested, based on the candidates with the lowest dissimilarity score. For a discussion of setting the threshold as well as the statistical validity of the dissimilarity score and error rates, consult NIST Fingerprint Image Quality (NFIQ) on page 14.

`Compare()` takes two single views from two FMDs and returns a dissimilarity score indicating the quality of the match.

The majority of applications should use the `Identify` method to implement both identification and verification. However, in a few special cases, e.g., using multi-modal biometrics, or doing statistical risk assessment, the `Compare` method allows you to compare two FMVs to determine their actual degree of dissimilarity. This is useful for accuracy testing and diagnostics and is not intended to be used in final applications for actual fingerprint recognition. The `Compare` method returns a ***dissimilarity score*** with values:

- 0 = fingerprints are NOT dissimilar (i.e., they MATCH perfectly).
- `maxint` (#7FFFFFFF or 2147483647) = fingerprints are completely dissimilar (i.e., DO NOT match).
- Values close to 0 indicate very close matches, values closer to `maxint` indicate very poor matches.

The table below shows the relationship between the scores returned from `Compare` and the false match error rates observed in our test. The dissimilarity score distribution is estimated based on our internal testing, and may not be representative of the actual rate that will be observed in deployment.

Dissimilarity Score	False Match Rate
2147483	.1%
214748	.01%
21474	.001%
2147	.0001%

Function	Description
Compare	Compare two FMDs; supported formats are: Gold SDK, One Touch SDK, ANSI and ISO.
Identify	Identify an FMD: given an array of FMDs, this function returns an array of candidates that match the original fingerprint (an FMV within an FMD) within the threshold of error. Supported formats are: Gold SDK, One Touch SDK, ANSI and ISO.

Enrollment

`CreateEnrollmentFmd()` creates and returns an enrollment FMD. The method takes as input an enumeration which defines which format to use, e.g., ANSI or ISO. This method also takes as input an `IEnumerable<Fmd>` object type. See IEnumerables in the .NET Wrapper on page 33 for a brief description. As `IEnumerable<Fmd>` pertains to Enrollment, you may either send an array of FMDs to `CreateEnrollmentFmd()` because simple arrays satisfy the `IEnumerable<Fmd>` interface. You may also send a method which returns `IEnumerable<Fmd>`. Here is an example that uses an `IEnumerable<Fmd>` method. To keep the example simple, error checking is removed:

```
// Create an enrollment Fmd.
DataResult<Fmd> enrollmentResult = Enrollment.CreateEnrollmentFmd(
Constants.Formats.Fmd.ANSI,
CaptureExtractFmd()
);
```

Here is a function which captures and extracts an FMD, then returns `IEnumerable<Fmd>`: (the full version of this code is in the code sample)

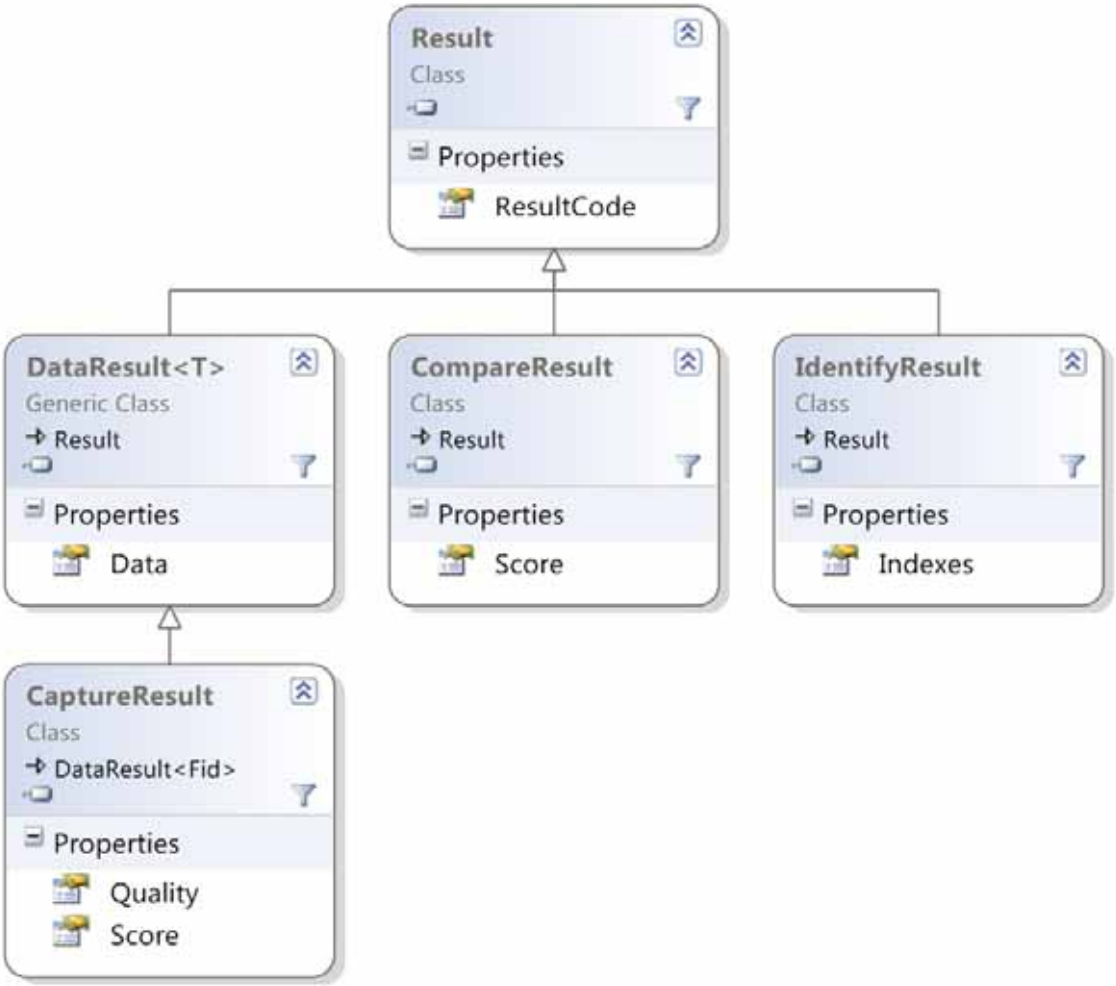
```
// Capture and extract an FMD and return as IEnumerable<Fmd>.
private IEnumerable<Fmd> CaptureAndExtractFmd()
{
  while (true)
  {
      // !!! Get Status and ensure that status is DP_STATUS_READY before continuing
      // Capture a fingerprint.
      CaptureResult captureResult = m_reader.Capture(
          Constants.Formats.Fid.ANSI,
          Constants.CaptureProcessing.DP_IMG_PROC_DEFAULT,
              /* Default processing used. */
          -1,
              /* No timeout. */
          reader.Capabilities.Resolutions[0]
              /* A resolution that is available to reader. */
          );

            // !!! Check for errors, use 'yield return null; or break;' to stop.
            yield return convertResult.Data;
  }
}
```

In VB.NET, the `IEnumerable<T>` class does not exist. The recommended method to enroll FMDs using VB.NET is to send `CreateEnrollmentFmd()` four captured FMDs in an array. If four FMDs are not enough, add a fifth captured and converted FMD to the array and send to `CreateEnrollmentFMD()`, and so on until an FMD is created.

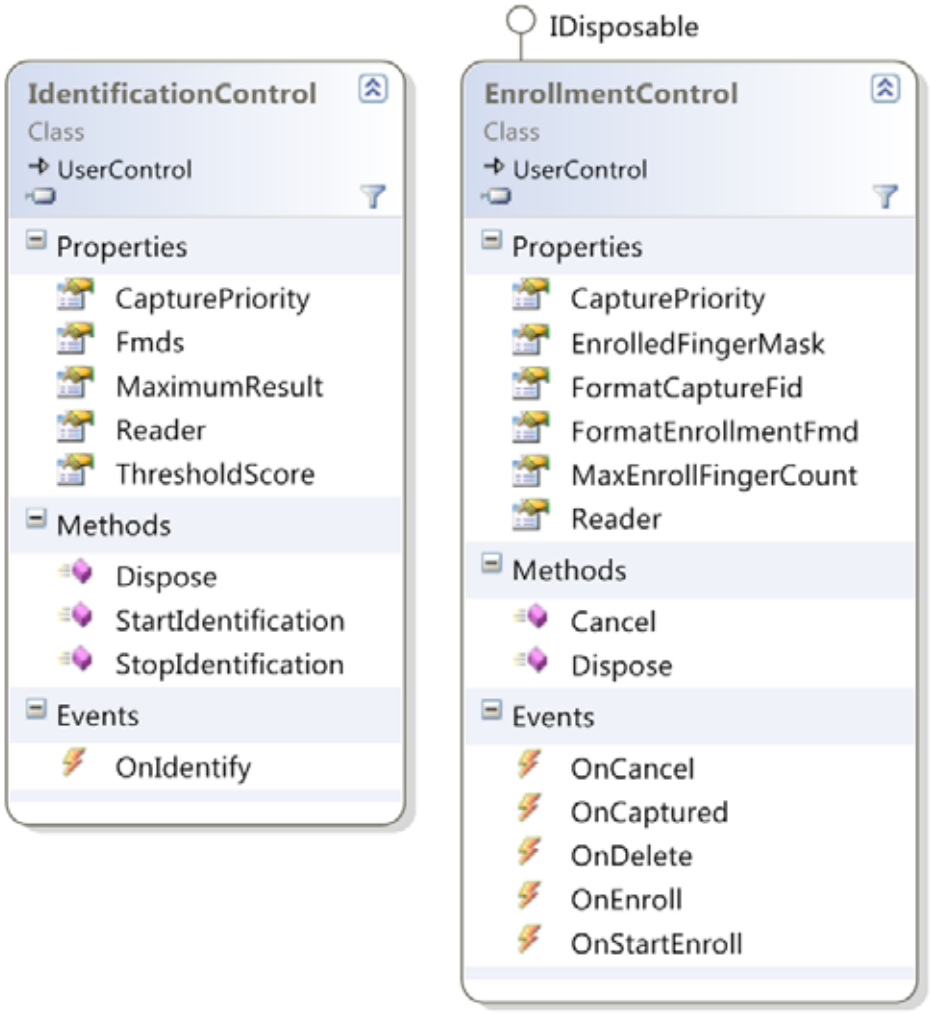
Common Data Structures for Results

The following classes are used when data is returned as a result of a U.are.U .NET operation.



Pre-Built Controls for Enrollment and Identification

In addition to the basic API, the SDK contains two pre-built controls that allow you to quickly add enrollment or identification into your application. These controls provide the same functionality as the DigitalPersona One Touch products.



THIS CHAPTER PROVIDES AN OVERVIEW OF THE API. FOR DETAILS OF USING THE API ON WINDOWS-BASED READERS, CONSULT THE U.ARE.U SDK PLATFORM GUIDE FOR WINDOWS.

The ActiveX API and controls are built as a wrapper to the C/C++ APIs, and is available for the Windows platform.

This chapter provides an overview of the API. For details of using the API on Windows-based readers, consult the U.are.U SDK Platform Guide for Windows.

Note that ActiveX is a Microsoft technology and is not supported on Mozilla Firefox and Google Chrome.

Importing the U.are.U ActiveX package

The U.are.U ActiveX library classes are aggregated into two DLLs:

- **DPXUru.dll** – ActiveX API library
- **DPCtlXUru.dll** – ActiveX GUI controls

Getting Detailed Documentation

This chapter provides an overview of the main methods in the ActiveX API. For a complete description of method parameters, the Doxygen documentation for the ActiveX libraries is provided. Consult the platform guide for details of where the Doxygen files are located.

Using the Package

Main Access Points

The main access point to the U.are.U ActiveX library is the `XReaderCollection` class.

- **To acquire a reference to XReaderCollection** use the `GetReaders()` method. To destroy `XReaderCollection`, (release all system resources associated with readers and make readers available for other processes) use the `Dispose()` method.
- **To acquire a reference(s) to individual readers**, use the `XReaderCollection` object, which is a collection of objects of type `XReader`.
- **To work with the FingerJet Engine, instantiate one of the classes:** `XComparison`, `XImporter`, `XFeatureExtraction` **or** `XEnrollment`. For example, to start enrolling fingerprints using the FingerJet engine, call the static method, `Enrollment.CreateEnrollmentFmd()` . This method takes as input an enumeration that specifies the format the enrollment template should be in, e.g., ANSI or ISO. This method also takes an `IEnumerable<Fmd>` object. See IEnumerables in the .NET Wrapper on page 33 for more information on `IEnumerable<Fmd>`.

SDKException

The `SDKException` class describes exceptions specific to the U.are.U SDK.

Serialization

The ActiveX API provides the ability to convert an FMD or FID into a format that can be read serially. Serialization allows you to easily transmit and store data as byte strings, streams or XML. This allows you to transmit the data or save it to standard file systems. The XML format can be used within HTML for building browser-based applications.

To return the data to its original format, *deserialization* is also provided.

Note that serialized FMDs and FIDs include the version number of the ActiveX API that was used for the serialization. If you attempt to deserialize with an older version of the API, the results may not be correct, so when deserializing, the API will throw an SDKException if the serialization was done using a later version of the wrapper.

Serialization in the API occurs using the default `System.Xml.Serialization.XmlSerializer`. The name of the root element of the generated XML is the same as the object type. Each public member of the object is represented as an XML element. Raw byte data is encoded as a Base-64 string by `XmlSerializer`.

Deserialization also uses `XmlSerializer`. The XML that was serialized is fed into `XmlSerializer` where the elements with values become object members with values of the same name as the element.

Working with Readers

Readers are accessed through the `XReaderCollection` object, which is a collection of `XReader` objects.

Each attached reader is represented with a `XReader` object. The `XReader` interface allows:

- Querying reader description and capabilities,
- Acquiring status of the reader,
- Capturing fingerprints,
- Starting and stopping image stream, and
- Resetting and calibrating the reader.

To acquire a reader, first instantiate a `XReaderCollection` object and call `XReaderCollection.GetReaders()` which returns the attached readers. Iterate through this list to look at the description object for the desired reader. Once finished with all of the `XReader` objects in the `XReaderCollection`, ensure that you call the `Dispose()` method to destroy the `XReaderCollection` object and release system resources.

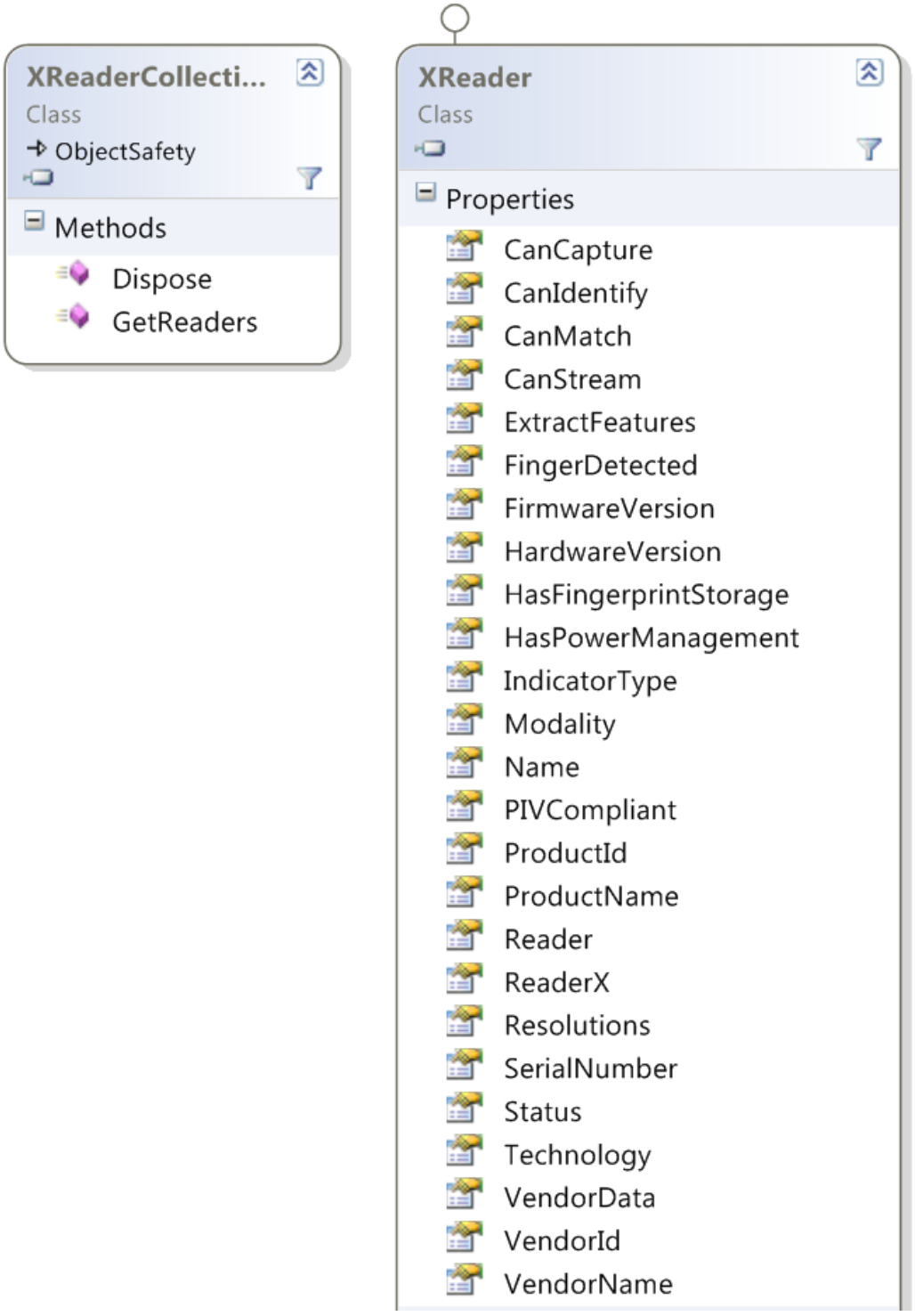
A Note About Internet Explorer and Process Merging

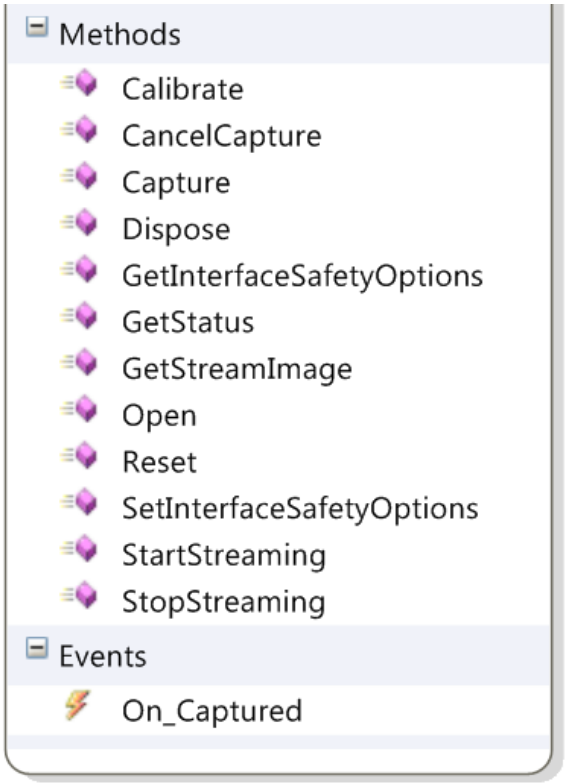
By default, Internet Explorer merges browser processes where it can, beginning with IE8. If your application is open in more than one browser window, users may experience erroneous behavior when your application calls `XReader.Dispose()` from one window.

To prevent this, users can use the **File > New Session** command to open a new window that does not merge processes with existing windows. You can also prevent IE from merging processes by using the **-nomerge** command line option when launching IE OR change the registry setting of `HKEY_CURRENT _USER\Software\Microsoft\Internet Explorer\Main` to set `SessionMerging` to 0. Note however that these options will affect IE's efficiency and startup time.

Class Diagrams

The UML diagrams below show the `XReaderCollection` and `XReader` classes.





The main reader hardware management methods in the `XReader` class are:

Function	Description
Open	Open a device and return the device capabilities. This method establishes an exclusive link to the device; no other processes will be able to use the device until you close it. The application *must* open the device before use.
GetStatus	Get the status for a device. You would normally check the device status before captures to ensure that the device is functioning and there are no error conditions. Returns an object of type `ReaderStatus` which describes the current status of the reader.
Calibrate	Calibrate a device. Some devices are self-calibrating. Ambient light or temperature can affect calibration, for some devices. Calibration can take several seconds.
Reset	Do a hardware reset on the reader. Hardware resets are typically needed only after a hardware problem (e.g., the device is unplugged or receives an electrostatic shock). Hardware resets typically only take a few milliseconds.
Dispose	Close device, release memory, remove child objects.

Capturing Fingerprints

The `Capture` function of the `XReader` class captures a fingerprint image for

- Enrollment (as part of the process described on page *12*)
- Identifying users with `Identify`
- Verifying a specific user identity with `Compare`

The primary fingerprint capture methods are:

Function	Description
Capture	Captures a fingerprint image from the open reader. This function signals the reader that a fingerprint is expected, and blocks until a fingerprint is received, capture fails or the reader times out.
CancelCapture	Cancel a pending capture

Streaming Fingerprints

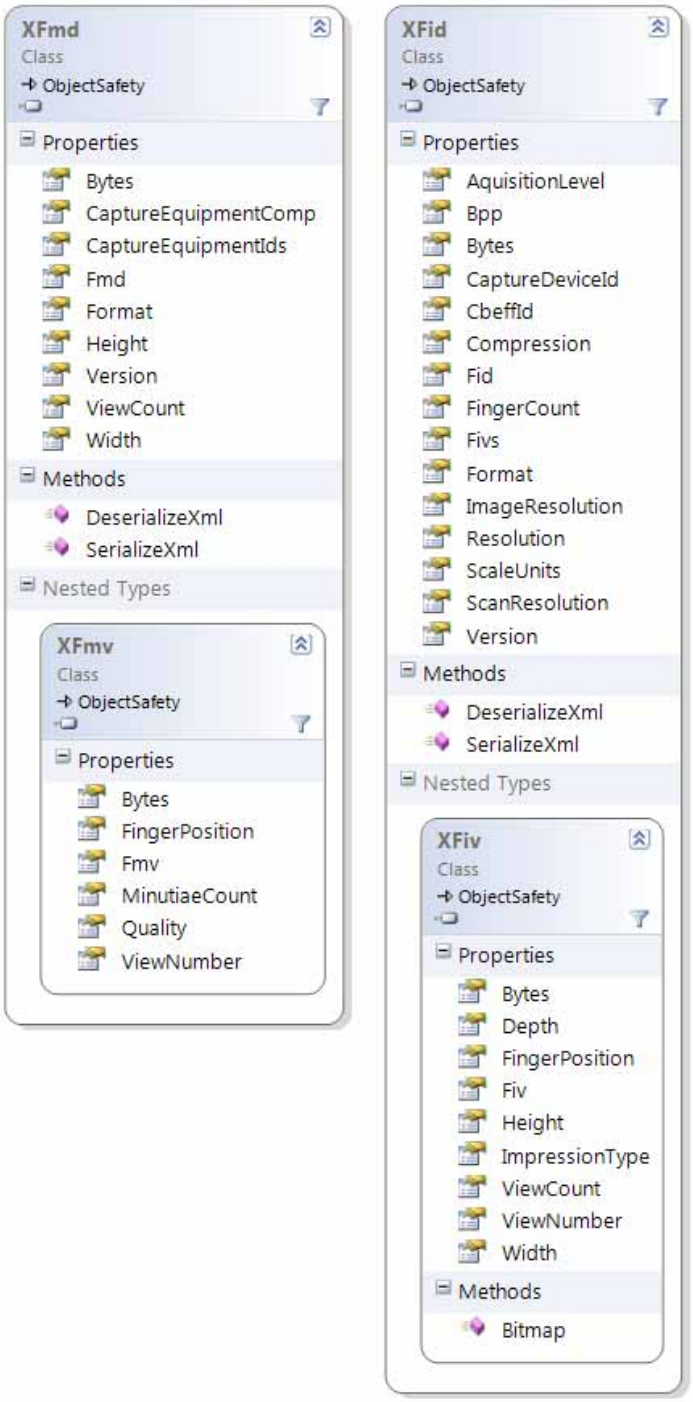
Not all readers support streaming mode. To determine if a specific reader supports this feature, check the value of the `XReader` property `CanStream`.

The streaming methods in `XReader` are:

Function	Description
StartStreaming	Put the reader into streaming mode. In this mode, the application must call `GetStreamImage()` to acquire images from the stream.
GetStreamImage	Capture a fingerprint image from the streaming data. After this function returns, the reader remains in streaming mode. Frame selection, scoring and other image processing are not performed by this function.
StopStreaming	End streaming mode

Managing Fingerprint Data

The ActiveX API implements fingerprint data as shown in the UML diagram below:



In addition to the usual methods for working with data, this class includes the methods below for serializing and deserializing (as described in Importing the U.are.U .NET package on page 32).

Function	Description
SerializeXml	Static method which serializes FMDs and FIDs into an XML string. Note that raw data is encoded as base-64.
DeserializeXml	Static method which de-serializes XML-encoded FMDs and FIDs back into a U.are.U FMD or FID data structure.

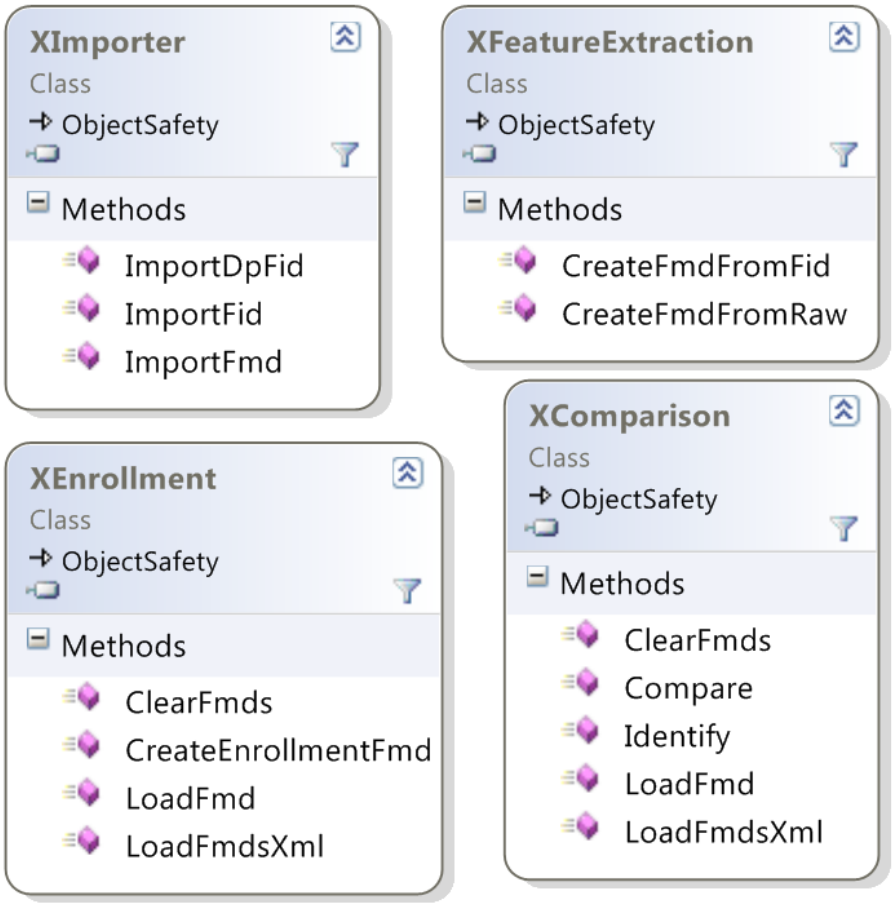
Accessing the FingerJet Engine

The FingerJet Engine interfaces provide functionality to

- Extract fingerprint features (create FMDs),
- Identify and compare FMDs, and
- Create enrollment FMDs.

Refer to Understanding the Data Flow on page 11 for a description of enrollment and comparison terminology and data flow.

The UML diagram below shows the relevant classes:



Creating FMDs from images

The `CreateFmdFromFid()` method can be used in two ways:

1 Extract fingerprint minutiae from a raw image and create an FMD.

2 Extract fingerprint minutiae from an FID and create an FMD.

The following limitations are applied to the raw images and FIDs:

- 8 bits per pixel
- no padding
- square pixels (horizontal and vertical dpi are the same)

The size of the resulting FMD will vary depending on the minutiae in a specific fingerprint.

Identification and Comparison

`Identify()` identifies a single FMD against an array of FMDs. This function takes as inputs:

- A single view in an FMD
- An array of FMDs (each FMD can contain up to 16 views) to compare
- The desired number of candidates to return
- The threshold for False Positive Identification Rate (FPIR) that is permitted

and returns matches as an array of integer pairs which provide the finger index and view index of each match.

Each time a view has a score lower than the threshold FPIR, that view is marked as a possible candidate. Then when all possible candidates are identified (i.e., they meet the threshold), they are ranked by their score. Finally, the function returns as many candidates as requested, based on the candidates with the lowest dissimilarity score. For a discussion of setting the threshold as well as the statistical validity of the dissimilarity score and error rates, consult NIST Fingerprint Image Quality (NFIQ) on page 14.

`Compare()`takes two single views from two FMDs and returns a dissimilarity score indicating the quality of the match.

The majority of applications should use the `Identify` method to implement both identification and verification. However, in a few special cases, e.g., using multi-modal biometrics, or doing statistical risk assessment, the `Compare` method allows you to compare two FMVs to determine their actual degree of dissimilarity. This is useful for accuracy testing and diagnostics and is not intended to be used in final applications for actual fingerprint recognition. The `Compare` method returns a ***dissimilarity score*** with values:

- 0 = fingerprints are NOT dissimilar (i.e., they MATCH perfectly).
- `maxint` (#7FFFFFFF or 2147483647) = fingerprints are completely dissimilar (i.e., DO NOT match).
- Values close to 0 indicate very close matches, values closer to `maxint` indicate very poor matches.

The table below shows the relationship between the scores returned from `Compare` and the false match error rates observed in our test. The dissimilarity score distribution is estimated based on our internal testing, and may not be representative of the actual rate that will be observed in deployment.

Dissimilarity Score	False Match Rate
2147483	.1%
214748	.01%
21474	.001%
2147	.0001%

Function	Description
Compare	Compare two FMDs; supported formats are: Gold SDK, One Touch SDK, ANSI and ISO.
Identify	Identify an FMD: given an array of FMDs, this function returns an array of candidates that match the original fingerprint (an FMV within an FMD) within the threshold of error. Supported formats are: Gold SDK, One Touch SDK, ANSI and ISO.

Enrollment

`CreateEnrollmentFmd()` creates and returns an enrollment FMD. The method takes as input:

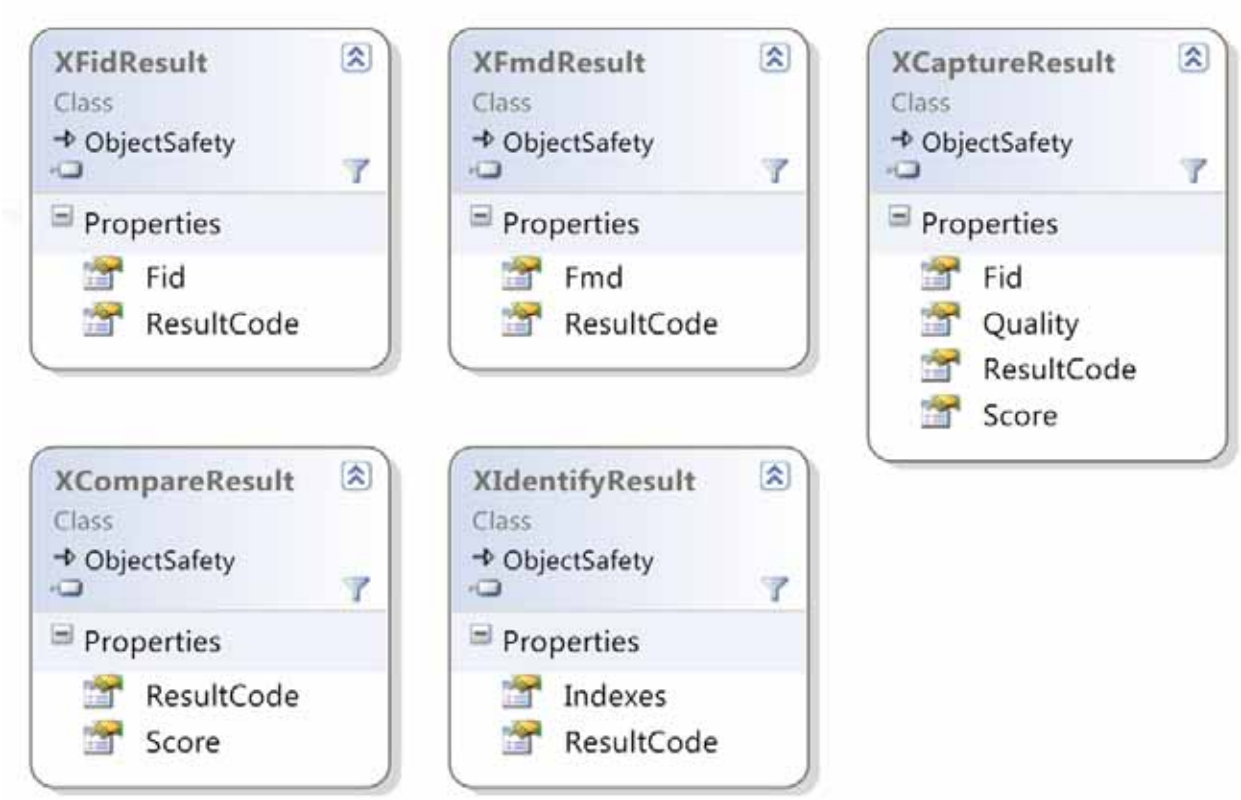
- Which format to use, e.g., ANSI or ISO
- An `ArrayList` object containing the FMDs to enroll

The recommended sequence in which to enroll FMDs is to send `CreateEnrollmentFmd()` four captured FMDs in an array. If four FMDs are not enough, you can capture an FID, convert it to an additional FMD, add it to the array

and call `CreateEnrollmentFMD()` again. If `CreateEnrollmentFMD()`, fails again, you can continue to add FMDs to the array and call `CreateEnrollmentFMD()` until an enrollment FMD is created successfully.

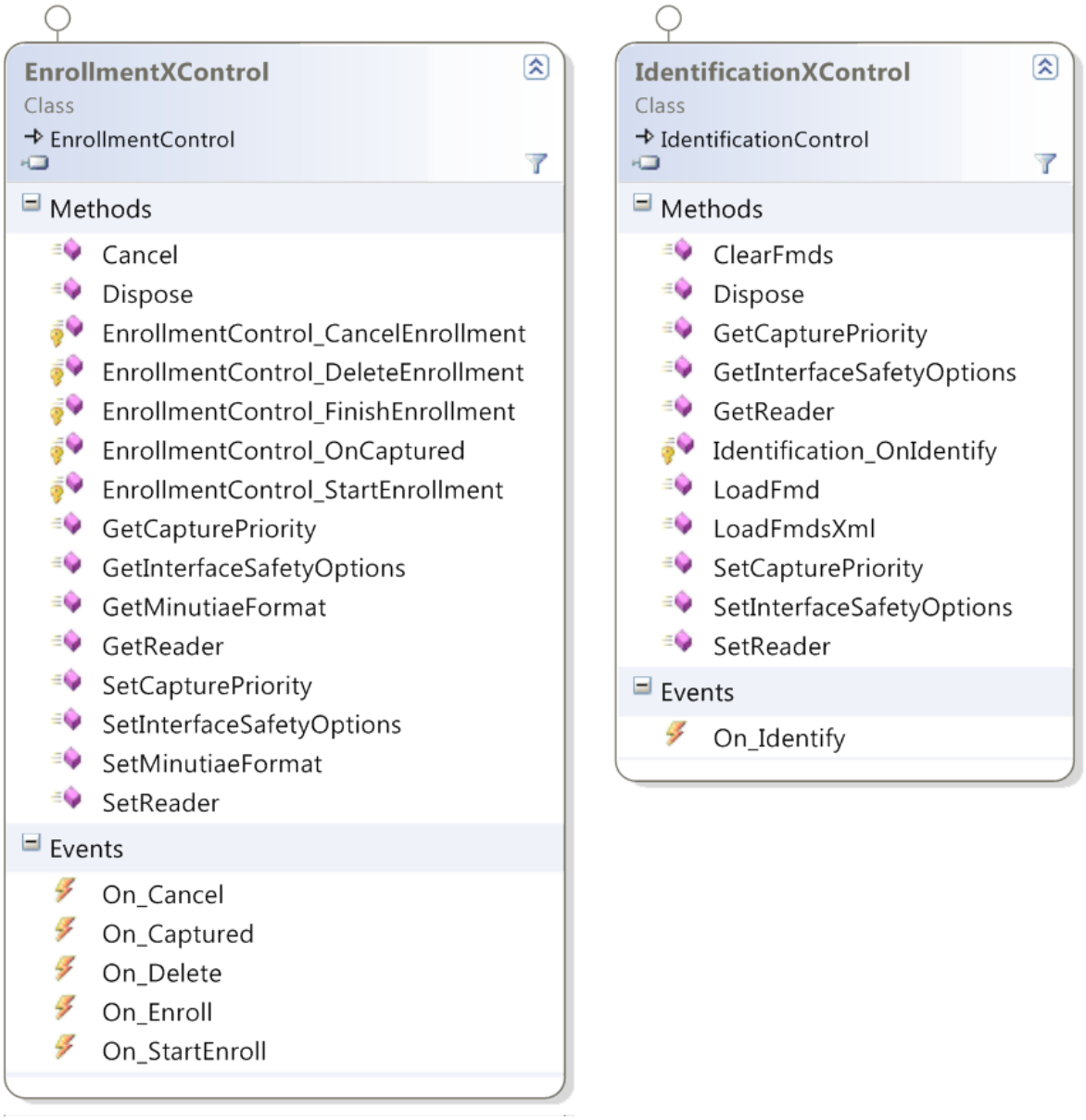
Common Data Structures for Results

The following classes are used when data is returned as a result of a U.are.U ActiveX operation.



Pre-Built Controls for Enrollment and Identification

In addition to the basic API, the SDK contains two pre-built controls that allow you to quickly add enrollment or identification into your application. These controls provide the same functionality as the DigitalPersona One Touch products.



JavaPOS

THIS CHAPTER DESCRIBES HOW TO USE THE JAVAPOS API.

The U.are.U JavaPOS API is built on the U.are.U Java API. This JavaPOS API is geared for Point of Sale applications and has the following features:

- **Fully compliant with JavaPOS 1.13.** The U.are.U JavaPOS API conforms to the specifications for the Biometrics device category in Chapter 5, "Biometrics," of the *UnifiedPOS Retail Peripheral Architecture*, Version 1.13 (July 15, 2009).

  The complete UPOS documentation is available at http://www.nrf-arts.org/UnifiedPOS/default.htm.
- **Backward compatible with previous DigitalPersona JavaPOS product (U.are.U UPOS for JavaPOS SDK),** with only a few caveats. This new API is the result of merging the previous JavaPOS SDK with the U.are.U SDK - the internal architecture has been completely rewritten. For users of the previous SDK, the updated SDK means an upgraded internal architecture for more robust performance -- up to ten times faster for identification. If you are upgrading an existing application, see Upgrading from U.are.U UPOS for OPOS/JavaPOS on page 20 for more details.
- **In addition to the JavaPOS API, the U.are.U SDK includes a JavaPOS Device Service object**, which can be used with any JavaPOS Device Control for the Biometrics device category.
- **Because the JavaPOS API is built as an adjunct to the Java API, applications can use both the JavaPOS standard operations AND use the U.are.U Java API** (described in The JAVA API on page 28). Java methods can be used to access streaming features or to import data.

Terminology Note

The U.are.U JavaPOS API conforms to the standard terminology used by the JavaPOS spec from 2009 whereas the other APIs in the U.are.U SDK generally use terminology that matches evolving industry standards. If you are going to use the U.are.U Java API along with JavaPOS, you need to note that in JavaPOS, the extracted fingerprint data (template) is stored in a biometric information record (BIR). The templates created through enrollment and capture are equivalent to Fingerprint Minutiae Data (FMD) records in the other APIs of the U.are.U SDK. Note that JavaPOS templates include a 10- or 45-byte JavaPOS header in addition to the data itself.

Working with Fingerprint Data in JavaPOS

The U.are.U JavaPOS API provides two types of fingerprint data: raw images and fingerprint templates. The data flow for JavaPOS applications is the same as described in the topic Understanding the Data Flow on page 11 except that JavaPOS data has additional JavaPOS headers and JavaPOS does not use the FID/FMD terminology used by U.are.U. Note that there is no easy method provided in the API for converting data from JavaPOS BIRs to FMDs.

Fingerprint Data for Raw Images (Captures)

Per the JavaPOS spec, raw fingerprint scans (images) are available in the **RawSensorData** property when **StatusUpdateEvent** triggers.

Fingerprint Data for Captures and Enrollment Templates (BIRs)

The U.are.U JavaPOS API now supports four different JavaPOS biometric information record (BIR) formats, as set by the *Algorithm* property:

Function	Description
0	Default (Same as 1)
1	DigitalPersona format with 45-byte header that conforms to JavaPOS 1.13 spec. New captures and enrollments will be done in the DigitalPersona format with 45-byte headers. Verify and identify will correctly compare with BIRs that have 45-byte headers, the previous 10-byte headers or a mixture.
2	ANSI INSITS 378-2004. New captures and enrollments will be created in ANSI format with a 45-byte JavaPOS header. Verify and identify will require that all BIRs be in ANSI format.
3	ISO/IEC 19794-2:2005. New captures and enrollments will be created in ISO format with a 45-byte JavaPOS header. Verify and identify will require that all BIRs be in ISO format.
4	JavaPOS 1.x compatible. This is the previous DigitalPersona format with 10-byte header. This format was the default (and only) format used in the U.are.U UPOS for JavaPOS SDK. New captures and enrollments will be created in the DigitalPersona format with 10-byte headers. Verify and identify will correctly compare with BIRs that have 10-byte headers.

Identification and verification require that all BIRs be in the same format. Stored enrollment BIRs can be in four formats:

1 DigitalPersona format (10-byte headers, 45-byte headers or a mixture),

2 ANSI (with 45-byte headers)

3 ISO (with 45-byte headers)

4 JavaPOS 1.x compatible (DigitalPersona format with 10-byte headers).

For example, if the *Algorithm* property is set to ANSI and you receive ANSI captures, those fingerprints cannot be matched against templates that are stored in a DigitalPersona format.

The value of the *Algorithm* property must be set before the device is enabled (i.e., after the device is opened and claimed, but before it is enabled).

Working with DigitalPersona Record Formats

The format for the new default 45-byte headers is provided in the JavaPOS spec. The first 10 bytes of the 45-byte header are the same as the previous 10-byte header except for the header version number.

In both 10-byte and 45-byte headers, the 5th byte specifies the header format:

- The 5th byte is 0x10 for a 10-byte header
- The 5th byte is 0x20 for a 45-byte header

Converting to a Different Data Format

If you wish to convert to a different data format, you must re-enroll users with the new format. There is no way to convert data between formats.

Getting Device-Specific Information with DirectIOEvent

The **DirectIOEvent** event is fired by a Service Object to deliver vendor-specific events to the application. The U.are.U JavaPOS API uses DirectIOEvent events to notify the application about device connection and disconnection and intermediate events such as finger touch or removal.

Syntax

upos::events::DirectIOEvent
EventNumber: int32 { read-only }
Data: int32 { read-write }
Obj: object { read-write }

Properties

This event contains the following attributes:

Property	Type	Description
EventNumber	*int*	Constants whose specific values are assigned by the Service Object
Data	*int*	Not used
Obj	*object*	Name of the fingerprint reader

EventNumber Return Values

These constants are defined in the `com.digitalpersona.javapos.services.biometrics.dpfpconstants` class.

Event Number	Description
DP_EVENT_DISCONNECT	The fingerprint reader was disconnected.
DP_EVENT_RECONNECT	The fingerprint reader was reconnected.
DP_EVENT_FINGER_TOUCHED	The fingerprint reader was touched.
DP_EVENT_FINGER_GONE	The finger was removed from the fingerprint reader.
DP_EVENT_COMPLETED	Capture completed.
DP_EVENT_IMAGE_READY	Raw image (from a capture) is available.
DP_EVENT_SAMPLE_QUALITY	Information about quality of the sample is available.

Implementation Notes

The following table provides information about how UPOS and JavaPOS properties, methods, and events are implemented in the U.are.U JavaPOS API.

Property	Type	Description
UPOS Common Properties		
AutoDisable	No	This property is initialized to **false** in the **open** method. Changing its value will have no effect.
CapCompareFirmwareVersion	Yes	This property is initialized to **false** in the **open** method since firmware comparison is not supported.
CapPowerReporting	Yes	This property is initialized to **JPOS_PR_NONE** in the **open** method since power reporting is not supported.
CapStatisticsReporting	Yes	This property is initialized to **false** in the **open** method since statistics reporting is not supported.
CapUpdateFirmware	Yes	This property is initialized to **false** in the **open** method since firmware updating is not supported.
CapUpdateStatistics	Yes	This property is initialized to **false** in the **open** method since statistics updating is not supported.
CheckHealthText	Yes	This property is initialized to an **empty string** in the **open** method. It is never modified since device health checking is not supported.
Claimed	Yes	
DataCount	Yes	
DataEventEnabled	Yes	Has value **true** when an enroll or capture or verify capture is in progress.
DeviceEnabled	Yes	Has value **true** when the service will return device events. The Algorithm property cannot be changed when the device is enabled.
DeviceServiceDescription	Yes	This property is set to “U.are.U Biometrics Device Service for JavaPOS, (c) 2007-2012 DigitalPersona, Inc.”
DeviceServiceVersion	Yes	This property is set to 1013000.
FreezeEvents	Yes	
PhysicalDeviceDescription	Yes	This property contains the product name, serial number, and vendor name for the DigitalPersona device.
PhysicalDeviceName	Yes	This property contains the product name of the DigitalPersona device.
PowerNotify	Yes	This property is initialized to **JPOS_PN_DISABLED** in the **open** method since power reporting is not supported.
PowerState	Yes	This property is initialized to **JPOS_PS_UNKNOWN** in the **open** method since power reporting is not supported.
State	Yes	
Specific Properties		
Algorithm	Yes	Specifies the data format used. See Fingerprint Data for Captures and Enrollment Templates (BIRs) on page 52 for details.
AlgorithmList	Yes	See Fingerprint Data for Captures and Enrollment Templates (BIRs) on page 52 for details.
BIR	Yes	

Property	Type	Description
CapPrematchData	Yes	This property is initialized to **false** in the **open** method since pre-matching is not supported.
CapRawSensorData	Yes	
CapRealTimeData	Yes	This property is initialized to **false** in the **open** method.
CapSensorColor	Yes	This property is initialized to **BIO_CSC_GRAYSCALE** in the **open** method.
CapSensorOrientation	Yes	This property is initialized to **BIO_CSO_NORMAL** in the **open** method.
CapSensorType	Yes	This property is initialized to **BIO_CST_FINGERPRINT** in the **open** method.
CapTemplateAdaptation	Yes	This property is initialized to **false** in the **open** method since template adaptation is not supported.
RawSensorData	Yes	Unkown OR contains raw sensor data. The data in this property exists when **CapRawSensorData** is set to **true**. and **StatusUpdateEvent** triggers.
RealTimeDataEnabled	Yes	This property is initialized to **false** in the **open**.
SensorBPP	Yes	This property is initialized to **8** in the **open** method. No other value is supported.
SensorColor	Yes	This property is initialized to **BIO_CSC_GRAYSCALE** in the **open** method. No other value is supported.
SensorHeight	Yes	This property is initialized to **0** in the **open** method. No other value is supported.
SensorOrientation	Yes	This property is initialized to **BIO_SO_NORMAL** in the **open** method. No other value is supported.
SensorType	Yes	This property is initialized to **BIO_ST_FINGERPRINT** in the **open** method.
SensorWidth	Yes	This property is initialized to **0** in the **open** method. No other value is supported.
UPOS Common Methods		
Open	Yes	
Close	Yes	
ClaimDevice	Yes	
ReleaseDevice	Yes	
CheckHealth	Yes	Only internal health check is supported. Attempts to perform an external or interactive check cause an exception to be thrown.
ClearInput	Yes	
ClearInputProperties	Yes	
ClearOutput	No	This method is not supported by the Biometrics device category.
DirectIO	Partial	An exception is thrown since device direct I/O is not supported.
CompareFirmwareVersion	Yes	An exception is thrown since **CapCompareFirmwareVersion** is set to false.
ResetStatistics	Yes	An exception is thrown since **CapUpdateStatistics** is set to false.
RetrieveStatistics	Yes	An exception is thrown since **CapStatisticsReporting** is set to false.
UpdateFirmware	Yes	An exception is thrown since **CapUpdateFirmware** is set to false.

Property	Type	Description
UpdateStatistics	Yes	An exception is thrown since **CapUpdateStatistics** is set to false.
Specific Methods		
beginEnrollCapture	Yes	
beginVerifyCapture	Yes	
endCapture	Yes	
identify	Yes	
identifyMatch	Yes	
processPrematchData	Yes	An exception is thrown since **CapPrematchData** is set to false.
verify	Yes	
verifyMatch	Yes	
UPOS Events		
DataEvent	Yes	The following event types are supported: BIO_DATA_ENROLL - Enrollment template created BIO_DATA_VERIFY - Verification template created
DirectIOEvent	Yes	See Getting Device-Specific Information with DirectIOEvent on page 54 for details.
ErrorEvent	Yes	**resultCode** - Standard JavaPOS Result Code **resultCodeExtended** - DigitalPersona Result Code - see Device-Related Error Codes on page 57 for details **errorLocus** - EL_INPUT **errorResponse** - ER_CLEAR
OutputCompleteEvent	No	This event is not supported by the Biometrics device category.
StatusUpdateEvent	Yes	Only occurs when **RawSensorData** has received a raw image from a capture.

Exceptions

The files `jpos.Const.*` contain the JavaPOS standard exception codes. The following exceptions are thrown by the U.are.U JavaPOS API:

- JPOS_E_FAILURE
- JPOS_E_ILLEGAL
- JPOS_E_NOHARDWARE
- JPOS_E_TIMEOUT

Device-Related Error Codes

For the device-related result codes returned by *EventNumber* after the **DirectIOEvent** event, see EventNumber Return Values on page 54.

The following error codes are returned in the *ResultCodeExtended* property of the **ErrorEvent** object.

IMPORTANT: You should always use the names of the return codes, such as FT_OK, rather than the numeric values because these values may change at any time, without notice. The numbers are provided as a reference for the error codes in the sample application.

Property	Type	Description
-1	FT_ERR_NO_INIT	The fingerprint feature extraction module or the fingerprint comparison module is not initialized.
-2	FT_ERR_INVALID_PARAM	One or more parameters are not valid.
-4	FT_ERR_IO	A generic I/O file error occurred.
-7	FT_ERR_NO_MEMORY	There is not enough memory to perform the action.
-8	FT_ERR_INTERNAL	An unknown internal error occurred.
-10	FT_ERR_UNKNOWN_DEVICE	The requested device is not known.
-11	FT_ERR_INVALID_BUFFER	A buffer is not valid.
-33	FT_ERR_UNKNOWN_EXCEPTION	An unknown exception occurred.

THIS CHAPTER DESCRIBES HOW TO USE THE OPOS API.

The U.are.U OPOS API is built on the U.are.U C/C++ API. The OPOS API is geared for Point of Sale applications and has the following features:

- **Fully compliant with OPOS 1.13.** The U.are.U OPOS API conforms to the specifications for the Biometrics device category in Chapter 5, “Biometrics,” of the *UnifiedPOS Retail Peripheral Architecture*, Version 1.13 (July 15, 2009). The complete UPOS documentation is available at *http://www.nrf-arts.org/UnifiedPOS/default.htm*.
- **Backward compatible with previous DigitalPersona OPOS product (U.are.U UPOS for OPOS SDK),** with a few minor caveats. This new API is the result of merging the previous OPOS SDK with the U.are.U SDK. For users of the previous SDK, the updated SDK means an upgraded internal architecture for more robust performance -- up to ten times faster for identification. If you are upgrading an existing application, see Upgrading from U.are.U UPOS for OPOS/JavaPOS on page 20 for more details.
- **The OPOS API is a set of COM objects** that acts as an extension to the DigitalPersona fingerprint reader driver, providing an OPOS-compliant application interface to DigitalPersona products. It consists of a Control Object (CO) (an ActiveX® Control) for the OPOS Biometrics device category and a Service Object (SO).
- **Because the OPOS API is built as an adjunct to the C/C++ API, applications can use both the OPOS standard operations AND use the U.are.U C/C++ API** (described in The C/C++ APIs on page 23). C/C++ methods can be used to access streaming features or to import data.
- A complete reference to the controls, properties, methods and events of the OPOS interface is provided in Appendix A of the UnifiedPOS specification, a PDF document titled “UnifiedPOS Retail Peripheral Architecture.” Download the current version from *http://www.nrf-arts.org/content/unifiedpos*.

Terminology Note

The U.are.U OPOS API conforms to the standard terminology used by the OPOS spec from 2009 whereas the other APIs in the U.are.U SDK generally use terminology that matches evolving industry standards. If you are going to use the U.are.U C/C++ API along with OPOS, you need to note that in OPOS, the extracted fingerprint data (template) is stored in a biometric information record (BIR). The templates created through enrollment and capture are equivalent to Fingerprint Minutiae Data (FMD) records in the other APIs of the U.are.U SDK. Note that OPOS templates include a 12- or 45-byte OPOS header in addition to the data itself.

Working with Fingerprint Data in OPOS

The U.are.U OPOS API provides two types of fingerprint data: raw images and fingerprint templates. The data flow for OPOS applications is the same as described in Understanding the Data Flow on page 11 except that OPOS data has additional OPOS headers and OPOS does not use the FID/FMD terminology used by U.are.U. Note that there is no easy method provided in the API for converting data from OPOS BIRs to FMDs.

Fingerprint Data for Raw Images (Captures)

Per the OPOS spec, raw fingerprint scans (images) are available in the **RawSensorData** property when **StatusUpdateEvent** triggers.

Fingerprint Data for Captures and Enrollment Templates (BIRs)

The U.are.U OPOS API now supports four different OPOS biometric information record (BIR) formats, as set by the *Algorithm* property:

Value	Meaning
0	Default (Same as 1)
1	DigitalPersona format with 45-byte header that conforms to OPOS 1.13 spec. New captures and enrollments will be done in DigitalPersona format with 45-byte headers. Verify and identify will correctly compare with BIRs that have 45-byte headers, the previous 12-byte headers or a mixture.
2	ANSI INSITS 378-2004. New captures and enrollments will be created in ANSI format with a 45-byte OPOS header. Verify and identify will require that all BIRs be in ANSI format.
3	ISO/IEC 19794-2:2005. New captures and enrollments will be created in ISO format with a 45-byte OPOS header. Verify and identify will require that all BIRs be in ISO format.
4	OPOS 1.x compatible. This is the previous DigitalPersona format with 12-byte header. This format was the default (and only) format used in the U.are.U UPOS for OPOS SDK. New captures and enrollments will be created in the DigitalPersona format with 12-byte headers. Verify and identify will correctly compare with BIRs that have 12-byte headers.

Identification and verification require that all BIRs be in the same format. The BIRs can be in four formats:

1 DigitalPersona format (12-byte headers, 45-byte headers or a mixture),

2 ANSI (with 45-byte headers)

3 ISO (with 45-byte headers)

4 OPOS 1.x compatible (DigitalPersona format with 12-byte headers).

For example, if the *Algorithm* property is set to ANSI and you receive ANSI captures, those fingerprints cannot be matched against templates that are stored in a DigitalPersona format.

The value of the *Algorithm* property must be set before the device is enabled (i.e., after the device is opened and claimed, but before it is enabled).

Working with DigitalPersona Record Formats

The format for the new default 45-byte headers is provided in the OPOS spec. The first 12 bytes of the 45-byte header are the same as the previous 12-byte header except for the header version number.

In both 12-byte and 45-byte headers, the 5th byte specifies the header format:

- The 5th byte is 0x10 for a 12-byte header
- The 5th byte is 0x20 for a 45-byte header

Converting to a Different Data Format

If you wish to convert to a different data format, you must re-enroll users with the new format. There is no way to convert data between formats.

Getting Device-Specific Information with DirectIOEvent

The **DirectIOEvent** event is fired by a Service Object to deliver vendor-specific events to the application. The U.are.U JavaPOS API uses DirectIOEvent events to notify the application about device connection and disconnection and intermediate events such as finger touch or removal.

This chapter contains specific information about the U.are.U®SDK implementation of the OPOS Control.

DataEvent

Syntax

```
<< event >> upos::events::DataEvent
                Status: int32 { read-only }
```

Description

This event is fired to provide input data from the fingerprint reader to the application. The actual input data is placed in one or more device-specific properties.

Attribute

This event contains the following attribute:

Attribute	Type	Description
Status	*int32*	BIO_DATA_ENROLL if enroll capture is completed. BIO_DATA_VERIFY if verify capture is completed.

DirectIOEvent

Syntax

```
<< event >> upos::events::DirectIOEvent
                EventNumber: int32 { read-only }
                Data: int32 { read-write }
                Obj: object { read-write }
```

Description

This event is fired by the Service Object (SO) to communicate directly with the application. **DirectIOEvent** is used in the U.are.U®SDK to notify the user of the image-capturing status, fingerprint reader connection status, and image quality, etc., whenever required.

Attributes

This event contains the following attributes:

Attribute	Type	Description
EventNumber	*int32*	Event number whose specific values are assigned by the SO.
Data	*int32*	Additional numeric data. Specific values vary by the *EventNumber* and the SO.
Obj	*object*	Additional data whose use varies by the *EventNumber* and the SO.

EventNumber Return Values

Event Number	Description
1	The fingerprint reader was disconnected.
2	The fingerprint reader was reconnected.
3	The fingerprint reader was touched.
4	The finger was removed from the fingerprint reader.
5	A fingerprint image is ready for processing.
6	Provides information about the quality of the fingerprint image.
7	A supplied fingerprint credential was added to the fingerprint enrollment template.
8	The fingerprint capture operation was stopped.

Data Return Values

Event Number[a]	Data	Description
6	DP_QUALITY_GOOD, DP_QUALITY_NONE, etc.	Can be any value from the DP_SAMPLE_QUALITY enumeration in the table on the next page.
7	1, 2, 3, etc.	Holds the number of images added.

a. For every other *EventNumber* (1 through 5 and 8), *Data* holds the value 0 (Not Supported).

DP_SAMPLE_QUALITY Enumeration

Value	Meaning
DP_QUALITY_GOOD (0)	The image is of good quality.
DP_QUALITY_NONE (1)	There is no image.
DP_QUALITY_TOOLIGHT (2)	The image is too light.
DP_QUALITY_TOODARK (3)	The image is too dark.
DP_QUALITY_TOONOISY (4)	The image is too noisy.
DP_QUALITY_LOWCONTR (5)	The image contrast is too low.
DP_QUALITY_FTRNOTENOUGH (6)	The image does not contain enough information.
DP_QUALITY_NOCENTRAL (7)	The image is not centered.

Obj Return Values

Event Number	Object
1	The fingerprint reader was disconnected.
2	The fingerprint reader was reconnected.
3	The fingerprint reader was touched.
4	The finger was removed from the fingerprint reader.
5	NULL (Not Supported).
6	Provides information about the quality of the fingerprint image.
7	A supplied fingerprint credential was added to the fingerprint enrollment template.
8	The fingerprint capture operation was stopped.

StatusUpdateEvent

Syntax

<< event >> upos::events::StatusUpdateEvent
Status: int32 { read-only }

Description

This event is used in the U.are.U®SDK to notify the user that a raw image is available for use.

Attribute

This event contains the following attribute:

Attribute	Type	Description
Status	*int32*	The *Status* parameter notifies the user that raw image data is available.

Status Return Values

StatusUpdateEvent	Value	Meaning
1	BIO_SUE_RAW_DATA	Raw image data is available.

Implementation Notes

The following table provides information about how UPOS properties, methods, and events are implemented in the DigitalPersona U.are.U®SDK.

Name	Implemented[a]	Comment
UPOS Common Properties		
AutoDisable	Partial	This property is initialized to false in the **open** method. It is not modified during execution of the application since it is not required to automatically disable the device when data is received.
BinaryConversion	Partial	This property is initialized to OPOS_BC_NONE in the **open** method and is not modified during execution of the application.
CapCompareFirmwareVersion	Partial	This property is initialized to false in the **open** method. It is not modified during execution of the application since firmware version comparison is not supported.
CapPowerReporting	Partial	This property is initialized to OPOS_PR_NONE in the **open** method. It is not modified during execution of the application since power reporting is not supported.
CapStatisticsReporting	Partial	This property is initialized to false in the **open** method. It is not modified during execution of the application since statistics reporting is not supported.
CapUpdateFirmware	Partial	This property is initialized to false in the **open** method. It is not modified during execution of the application since firmware updating is not supported.
CapUpdateStatistics	Partial	This property is initialized to false in the **open** method. It is not modified during execution of the application since statistics updating is not supported.
CheckHealthText	No	

Name	Implemented[a]	Comment
Claimed	Yes	This property is initialized to false in the **open** method. It is set to true on claim and set to false again on release.
ControlObjectDescription	Yes	
ControlObjectVersion	Yes	
DataCount	Partial	This property is initialized to 0 in the **open** method and is not modified during execution of the application.
DataEventEnabled	Partial	This property is initialized to false in the **open** method and is set to true after a successful claim operation.
DeviceDescription	Yes	
DeviceEnabled	Partial	This property is initialized to false in the **open** method and is set to true after a successful claim operation. Since the **AutoDisable** property is never set to true, **DeviceEnabled** always remains true.
DeviceName	Yes	
FreezeEvents	Partial	This property is initialized to false in the **open** method. It is not modified during execution of the application since it is not required to freeze events.
OpenResult	Yes	
OutputID	No	This property is not supported by the Biometrics device category.
PowerNotify	Partial	This property is initialized to OPOS_PN_DISABLED in the **open** method. It is not modified during execution of the application since power reporting is not supported.
PowerState	Partial	This property is initialized to OPOS_PS_UNKNOWN in the **open** method. It is not modified during execution of the application since power reporting is not supported.
ResultCode	Yes	
ResultCodeExtended	No	
ServiceObjectDescription	Yes	
ServiceObjectVersion	Yes	
State	Partial	This property is initialized to OPOS_S_IDLE in the **open** method and is set to OPOS_S_CLOSED in the **close** method.
Specific Properties		
Algorithm	Yes	See Fingerprint Data for Captures and Enrollment Templates (BIRs) on page 59 for details.
AlgorithmList	Yes	See Fingerprint Data for Captures and Enrollment Templates (BIRs) on page 59 for details.
BIR	Yes	This property holds the biometric data that is captured and returned to the application.
CapPrematchData	Partial	This property is initialized to false in the **open** method. It is not modified during execution of the application since the MOC (Match-on-Card) SmartCard technology needed to generate a processed **BIR** based on pre-match data stored on a SmartCard is not supported.
CapRawSensorData	Yes	This property is initialized to true in the **open** method and is not modified during execution of the application.
CapRealTimeData	Partial	This property is initialized to false in the **open** method and is not modified during execution of the application.

Name	Implemented[a]	Comment
CapSensorColor	Yes	This property is initialized to BIO_CSC_GRAYSCALE in the **open** method and is not modified during execution of the application.
CapSensorOrientation	Yes	This property is initialized to BIO_CSO_INVERTED in the **open** method and is not modified during execution of the application.
CapSensorType	Yes	This property is initialized to BIO_CST_FINGERPRINT in the **open** method and is not modified during execution of the application.
CapTemplateAdaptation	Partial	This property is initialized to false in the **open** method. It is not modified during the execution of the application since template adaptation is not supported.
RawSensorData	Yes	This property holds the raw biometric data that is captured and returned to the application.
RealTimeDataEnabled	Partial	This property is initialized to false in the **open** method and is not modified during execution of the application.
SensorBPP	Yes	This property is initialized by invoking the DigitalPersona function that defines the image resolution, in pixels.
SensorColor	Yes	This property is initialized by invoking the DigitalPersona function that defines the image type.
SensorHeight	Yes	This property is initialized by invoking the DigitalPersona function that defines the image height, in pixels.
SensorOrientation	Yes	This property is initialized to BIO_SO_INVERTED in the **open** method and is not modified during execution of the application.
SensorType	Yes	This property is initialized to BIO_ST_FINGERPRINT in the **open** method and is not modified during execution of the application.
SensorWidth	Yes	This property is initialized by invoking the DigitalPersona function that defines the image width, in pixels.
Common Methods		
Open	Yes	
Close	Yes	
ClaimDevice	Yes	
ReleaseDevice	Yes	
CheckHealth	No	
ClearInput	No	
ClearInputProperties	No	
ClearOutput	No	This method is not supported by the Biometrics device category.
DirectIO	No	
CompareFirmwareVersion	No	
ResetStatistics	No	
RetrieveStatistics	No	
UpdateFirmware	No	
UpdateStatistics	No	
Specific Methods		
beginEnrollCapture	Yes	
beginVerifyCapture	Yes	

Name	Implemented[a]	Comment
endCapture	Yes	
identify	Yes	
identifyMatch	Yes	
processPrematchData	No	
verify	Yes	
verifyMatch	Yes	
Events		
DataEvent	Yes	Valid values for *Status* parameter are 1- BIO_DATA_ENROLL if enroll capture is completed. 2- BIO_DATA_VERIFY if verify capture is completed.
DirectIOEvent	Specific	Return values are 1 - DP_DIOE_DISCONNECT 2 - DP_DIOE_RECONNECT 3 - DP_DIOE_FINGER_TOUCHED 4 - DP_DIOE_FINGER_GONE 5 - DP_DIOE_IMAGE_READY 6 - DP_DIOE_SAMPLE_QUALITY 7 - DP_DIOE_ENROLL_FEATURES_ADDED 8 - DP_DIOE_OPERATION_STOPPED
ErrorEvent	Yes	**ResultCode** - Standard OPOS Result Code
	Specific	**ResultCodeExtended** - DigitalPersona Result Code
	Yes	**ErrorLocus** - EL_INPUT
	Yes	**ErrorResponse** - ER_CLEAR
OutputCompleteEvent	No	This event is not supported by the Biometrics device category.
StatusUpdateEvent	Partial	Valid values for *Status* parameter are 1 - BIO_SUE_RAW_DATA

a. Yes = Fully implemented
No = Not implemented
Partial = Implemented, but not all features are available
Specific = Implemented with device-specific features or data

Exceptions

The files `jpos.Const.*` contain the JavaPOS standard exception codes. The following exceptions are thrown by the U.are.U JavaPOS API:

- JPOS_E_FAILURE
- JPOS_E_ILLEGAL
- JPOS_E_NOHARDWARE
- JPOS_E_TIMEOUT

Device-Related Error Codes

For the device-related result codes returned by *EventNumber* after the **DirectIOEvent** event, see EventNumber Return Values on page 62.

The following error codes are returned in the *ResultCodeExtended* property of the **ErrorEvent** object.

IMPORTANT: You should always use the names of the return codes, such as FT_OK, rather than the numeric values because these values may change at any time, without notice. The numbers are provided as a reference for the error codes in the sample application.

Value	Result Code	Meaning
-1	FT_ERR_NO_INIT	The fingerprint feature extraction module or the fingerprint comparison module is not initialized.
-2	FT_ERR_INVALID_PARAM	One or more parameters are not valid.
-4	FT_ERR_IO	A generic I/O file error occurred.
-7	FT_ERR_NO_MEMORY	There is not enough memory to perform the action.
-8	FT_ERR_INTERNAL	An unknown internal error occurred.
-10	FT_ERR_UNKNOWN_DEVICE	The requested device is not known.
-11	FT_ERR_INVALID_BUFFER	A buffer is not valid.
-33	FT_ERR_UNKNOWN_EXCEPTION	An unknown exception occurred.

Application Notes

THIS CHAPTER DESCRIBES SOME SUGGESTIONS FOR TROUBLESHOOTING YOUR APPLICATION.

General Fingerprint Issues

It is very important that you test your application with multiple and diverse people. The readability of fingerprints is affected by many factors including wear and tear, skin dryness, and age.

The middle finger often gives better scans than the index finger because the middle finger is typically less worn. We recommend that you enroll more than one finger for each user, preferably at least the index and middle fingers.

Minex Certification

The U.are.U SDK (2.3.0 and above) includes two versions of the FingerJet matching engine, versions 6 and 7.

Version 6 has a feature extractor certified by the National Institute of Standards and Technology's Minex program (*http://www.nist.gov/itl/iad/ig/minex.cfm*), as interoperable with other Minex-certified feature extractors and matchers.

Version 7 has both the feature extractor and matcher certified by the Minex program. However, the Minex-certified matcher is slower than the non-certified matcher in the version 6 FingerJet engine.

For customers who do not have a requirement to use a Minex-certified matcher, we recommend using the version 6 FingerJet engine. Specifically, for customers who use DigitalPersona legacy template formats, the version 7 FingerJet engine does not provide any improvements in matching performance over version 6.

C/C++ Issues

The `dpfj_identify` function returns `DPFJ_SUCCESS` if it is able to do identification (i.e., the FMDs are valid and correctly formed). However that does not mean that a candidate was found. You must check the number of returned candidates to see if any actual matches were found.

The `dpfj_compare` function returns `DPFJ_SUCCESS` if it is able to do the requested comparison (i.e., the FMDs are valid and correctly formed). However that does not mean that the fingerprints matched. To check whether they matched, you must look at the dissimilarity score (0=no match, `maxint`=perfect match, `maxint-threshold` = acceptable match).

General Terms

Fingerprint
The impression left from the friction ridges of a human finger.

Fingerprint characteristics
The biological finger surface details that can be detected and from which distinguishing and repeatable *fingerprint features* can be extracted.

Fingerprint minutiae
The *fingerprint characteristics* commonly used in fingerprint recognition systems. *Fingerprint minutiae* include:
Ridge ending – the abrupt end of a ridge
Ridge bifurcation – a single ridge that divides into two ridges
Short ridge, or independent ridge – a ridge that commences, travels a short distance and then ends
Island – a single small ridge inside a short ridge or ridge ending that is not connected to all other ridges
Ridge enclosure – a single ridge that bifurcates and reunites shortly afterward to continue as a single ridge
Spur – a bifurcation with a short ridge branching off a longer ridge
Crossover or bridge – a short ridge that runs between two parallel ridges
Delta – a Y-shaped ridge meeting
Core – a U-turn in the ridge pattern.

Fingerprint Data

Fingerprint sample
An analog or digital representation of a *fingerprint* obtained from a *fingerprint capture device*. See also *fingerprint image.*

Fingerprint image
A digital representation of a *fingerprint sample* encoded as a spatially mapped array of pixels. A *fingerprint image* is the only representation of a *fingerprint sample* supported by DigitalPersona products, thus the terms *fingerprint image* and *fingerprint sample* are used interchangeably.

Fingerprint features
The digital representation of *fingerprint characteristics*.

Fingerprint Image Data (FID)
The binary data containing one or more *fingerprint images* from one or multiple fingers of the same person.

Fingerprint Image View (FIV)
The part of an *FID* that contains a *fingerprint image* from a single impression of a single finger.
In the majority of cases, an *FID* contains only one *FIV*.

Fingerprint Minutiae Data (FMD)

The digital representation of *fingerprint minutiae*, and optionally other *fingerprint characteristics*, from one or multiple fingers of the same person.

Fingerprint Minutiae View (FMV)

The part of an *FMD* that contains *fingerprint features* from a single impression of a single finger.

Typically *FMDs* contain only one *FMV*.

Fingerprint template

The *fingerprint minutiae data* that is stored as a result of the *enrollment* process.

Fingerprint Devices

Fingerprint capture device

A device that collects a signal of *fingerprint characteristic* and outputs a *fingerprint sample.* This device can consist of one or more components, including hardware and supporting software. For example, a *fingerprint capture device* may include a camera, photographic paper, a printer, and/or a digital scanner.

Fingerprint reader

A *fingerprint capture device* that obtains a *fingerprint image* by direct interaction with a finger.

Fingerprint Recognition Terms

Capture

The process of acquiring a *fingerprint image* from a *fingerprint reader* or *fingerprint capture device.*

Enrollment

The process of *capturing* a *fingerprint image*(s) for an individual, extracting *fingerprint features,* optionally checking for duplicates, and storing the *fingerprint features* (*fingerprint template*). See also *FMD, Fingerprint Template*.

Comparison

The function which, given two *fingerprints* computes a *comparison score.*

Comparison score

A *comparison score* is a numerical value resulting from the comparison of the *fingerprint features* of two *fingerprints.*

Comparison scores are of two types: *similarity scores* and *dissimilarity scores*.

Comparison scores are calculated using algorithms that are specific to the fingerprint recognition system and, optionally, can be normalized in order to maintain a constant score distribution of *impostor verification attempts* or *open set identification attempts*.

Similarity Score

See *comparison score.*

Dissimilarity Score

See *comparison score.*

Fingerprint Recognition

Verification or *identification*.

Verification

The process of

1 *Capturing* a *fingerprint image* from an individual,

2 Extracting *fingerprint features,*

3 *Comparing* the captured image's *fingerprint features* with the *fingerprint template*(s) of the enrollee this individual claims to be and

4 Making the *match/non-match decision.*

Verification Threshold

The maximum degree of dissimilarity that is allowed between a *fingerprint image* and a *fingerprint template* in order to make the *match decision d*uring the *verification* process.

Match decision or match

A decision that two *fingerprints* are from the same finger (meet the *verification threshold*).

Non-match decision or non-match

A decision that two *fingerprints* are not from the same finger (do not meet the *verification threshold)*.

Identification

The process of

1 *Capturing* a *fingerprint image* from an individual,

2 Extracting *fingerprint features* and

3 *Comparing* the captured image's *fingerprint features* with the *fingerprint templates* from multiple individuals in order to create a candidate list of fingerprints that meet the *identification threshold*.

Identification Threshold

The maximum degree of dissimilarity that is allowed between a *fingerprint image* and a *fingerprint template* in order to call the *fingerprint template* a *candidate* in the *identification* process.

Candidate

A *fingerprint template* that is determined to be similar to a given *FMD* during identification.

Recognition Accuracy

The terminology below is used when discussing testing processes and reporting accuracy for fingerprint-enabled applications.

Genuine verification attempt

An attempt to compare *fingerprint features* with a *fingerprint template* where the *fingerprint features* and *fingerprint template* are from the same individual, and the *fingerprint features* and *fingerprint template* were captured at different times (recommended at least 2-3 weeks apart).

Impostor verification attempt

An attempt to compare *fingerprint features* with a *fingerprint template*, where the *fingerprint features* and *fingerprint template* are from two different people.

False match rate (FMR)

The ratio between the number of *impostor verification attempts* that produced a *match decision* and the total number of *impostor verification attempts. S*ee also *FAR.*

False non-match rate (FNMR)

The ratio between the number of *genuine verification attempts* that produced a *non-match decision* and the total number of *genuine verification attempts.* See also *FRR.*

False Accept Rate (FAR)

The application-specific measure of how often the application falsely grants a request.

In authentication applications, *FAR* is used in place of *FMR.*

False Reject Rate (FRR)

The application-specific measure of how often the application falsely rejects a request.

In authentication applications, *FRR* is used in place of *FNMR*.

Verification Detection Error Trade-off (DET) curve

A parametric curve that plots *FMR* on the x-axis and *FNMR* on the y-axis as a function of the *verification threshold*.

Probe

In simulation tests, a *fingerprint* used in searches (analogous to the user *fingerprint* used by a real application for searching).

Gallery

In simulation tests, a set of enrolled *fingerprints* (*fingerprint templates*) used for comparison purposes (analogous to the database of enrolled *fingerprint templates* used by a real application).

Open set identification attempt

For the purpose of testing, an attempt to compare a *probe* against a *gallery* that does not contain any impressions from any finger of the individual whose finger was used as the *probe*.

Closed set identification attempt

For the purpose of testing, an attempt to compare a *probe* against a *gallery*, where the *gallery* includes a *fingerprint template* from the same finger (from the same person) as used for the probe. The *probe* and the corresponding *fingerprint template* should be captured at different times (recommended at least 2-3 weeks apart).

False positive identification rate (FPIR)

Proportion of *identification* requests that return a *candidate* even though the person is not enrolled.

More precisely, for the purpose of recognition accuracy testing, the ratio between the number of *open set identification attempts* that returned one or more *candidates* and the total number of *open set identification attempts*.

False negative identification rate (FNIR)

Proportion of *identification* requests by enrollees that do not return the correct *candidate*.

More precisely, for the purpose of recognition accuracy testing, the ratio between the number of *closed set identification attempts* that did not return the correct *candidate* and the total number of *closed set identification attempts.*

Identification detection error trade-off (DET) curve

A parametric curve that plots *FPIR* on the x-axis and *FNIR* on the y-axis as a function of the *identification threshold*.

The *Identification DET curve* depends on the *gallery* size. Typically, multiple *identification DET curves* are shown on the same plot, one for each *gallery* size.

Index

A

Accessing the FingerJet Engine (ActiveX) **48**
Accessing the FingerJet Engine (Java API) **30**
Algorithm property **55, 64**
    allowed values **52, 59**
AlgorithmList property **55, 64**
ANSI INSITS 378-2004 **53, 60**
AutoDisable property **55, 63**

B

basics of fingerprint recognition **9**
BDB format **52, 59**
beginEnrollCapture method **57, 65**
beginVerifyCapture method **57, 65**
BinaryConversion property **63**
BIR format **52, 59**
BIR property **55, 64**

C

C/C++ APIs **23**
CapCompareFirmwareVersion property **55, 63**
CapPowerReporting property **55, 63**
CapPrematchData property **56, 64**
CapRawSensorData property **56, 64**
CapRealTimeData property **56, 64**
CapSensorColor property **56, 65**
CapSensorOrientation property **56, 65**
CapSensorType property **56, 65**
CapStatisticsReporting property **55, 63**
CapTemplateAdaptation property **56, 65**
Capturing Fingerprints **24**
Capturing Fingerprints (.NET API) **35**
CapUpdateFirmware property **55, 63**
CapUpdateStatistics property **55, 63**
CheckHealth method **56, 65**
CheckHealthText property **55, 63**
ClaimDevice method **56, 65**
Claimed property **55, 64**
Class Diagrams (ActiveX) **44**
ClearInput method **56, 65**
ClearInputProperties method **56, 65**
ClearOutput method **56, 65**
Close **56, 65**
Close method **56, 65**
common methods **56, 65**
    CheckHealth **56, 65**
    ClaimDevice **56, 65**
    ClearInput **56, 65**
    ClearInputProperties **56, 65**
    ClearOutput **56, 65**

CompareFirmwareVersion **56**, **65**
DirectIO **56**, **65**
Open **56**, **65**
ReleaseDevice **56**, **65**
ResetStatistics **56**, **65**
RetrieveStatistics **56**, **65**
UpdateFirmware **56**, **65**
UpdateStatistics **57**, **65**
common properties
AutoDisable **55**, **63**
BinaryConversion **63**
CapCompareFirmwareVersion **55**, **63**
CapPowerReporting **55**, **63**
CapStatisticsReporting **55**, **63**
CapUpdateFirmware **55**, **63**
CapUpdateStatistics **55**, **63**
CheckHealthText **55**, **63**
Claimed **55**, **64**
ControlObjectDescription **64**
ControlObjectVersion **64**
DataCount **55**, **64**
DataEventEnabled **55**, **64**
DeviceDescription **55**, **64**
DeviceEnabled **55**, **64**
DeviceName **64**
DeviceServiceVersion **55**
FreezeEvents **55**, **64**
OpenResult **64**
OutputID **64**
PhysicalDeviceDescription **55**
PhysicalDeviceName **55**
PowerNotify **55**, **64**
PowerState **55**, **64**
ResultCode **64**
ResultCodeExtended **64**
ServiceObjectDescription **64**
ServiceObjectVersion **64**
State **55**, **64**
CompareFirmwareVersion method **56**, **65**
Comparing Fingerprints (.NET API) **38**
ControlObjectDescription property **64**
ControlObjectVersion property **64**
Creating FMDs from images (.NET API) **38**
Creating FMDs from images (ActiveX) **48**
Creating FMDs from images (Java API) **30**

D

data formats and standards supported **8**
DataCount property **55**, **64**
DataEvent **57**, **66**
implementation notes for **61**
DataEventEnabled property **55**, **64**
DeviceDescription property **55**, **64**
DeviceEnabled property **55**, **64**
DeviceName property **64**

DeviceServiceVersion property **55**
DirectIO method **56**, **65**
DirectIOEvent **57**, **66**
    implementation notes **54**, **61**
    implementation notes for **61**
DP Capture API **23**

E

endCapture method **57**, **66**
Enrollment (.NET API) **39**
Enrollment (ActiveX) **49**
Enrollment (Java API) **31**
ErrorEvent **57**, **66**
    result codes returned in ResultCodeExtended property of **57**, **67**
events
    DataEvent **57**, **61**, **66**
    DirectIOEvent **57**, **61**, **66**
        implementation notes **54**, **61**
    ErrorEvent **57**, **66**
        result codes returned in ResultCodeExtended property of **57**, **67**
    OutputCompleteEvent **57**
    StatusUpdateEvent **52**, **57**, **59**, **63**, **66**
exceptions
    JPOS_E_FAILURE **57**, **66**
    JPOS_E_ILLEGAL **57**, **66**
    JPOS_E_NOHARDWARE **57**, **66**
    JPOS_E_TIMEOUT **57**, **66**
Extract FMD **25**

F

features of biometric technology **9**
FingerJet Engine **22**
FingerJet Engine API **24**
Fingerprint Capture Device Management **23**
fingerprint readers supported **17**
FreezeEvents property **55**, **64**

H

how this book is arranged **8**

I

Identification and Comparison (ActiveX) **49**
Identification and Comparison (Java API) **30**
Identify Fingerprint **25**
identify method **57**, **66**
identifyMatch method **57**, **66**
ISO/IEC 19794-2
    2005 **53**, **60**

J

JPOS_E_FAILURE **57**, **66**
JPOS_E_ILLEGAL **57**, **66**
JPOS_E_NOHARDWARE **57**, **66**
JPOS_E_TIMEOUT **57**, **66**

L

Library Management **23, 24**

M

Managing Fingerprint Data (.NET API) **36**
Managing Fingerprint Data (ActiveX) **47**
methods, common
    See common methods
methods, specific
    See specific methods

N

NIST Fingerprint Image Quality (NFIQ) **27**

O

Open method **56, 65**
OpenResult property **64**
OutputCompleteEvent **57**
    events
        OutputCompleteEvent **66**
OutputID property **64**

P

PhysicalDeviceDescription property **55**
PhysicalDeviceName property **55**
PowerNotify property **55, 64**
PowerState property **55, 64**
Pre-Built Controls for Enrollment and Identification (.NET API) **41**
Pre-Built Controls for Enrollment and Identification (ActiveX) **51**
processPrematchData method **57, 66**
properties, common
    See common properties
properties, specific
    See specific properties

R

RawSensorData property **56, 65**
RealTimeDataEnabled property **56, 65**
ReleaseDevice method **56, 65**
ResetStatistics method **56, 65**
ResultCode property **64**
ResultCodeExtended property **57, 64, 67**
RetrieveStatistics method **56, 65**

S

Select Fingerprint Matching Engine **24**
SensorBPP property **56, 65**
SensorColor property **56, 65**
SensorHeight property **56, 65**
SensorOrientation property **56, 65**
SensorType property **56, 65**
SensorWidth property **56, 65**
Serialization (.NET API) **33**
Serialization (ActiveX) **42**

ServiceObjectDescription property **64**
ServiceObjectVersion property **64**
specific methods
beginEnrollCapture **57**, **65**
beginVerifyCapture **57**, **65**
endCapture **57**, **66**
identify **57**, **66**
identifyMatch **57**, **66**
processPrematchData **57**, **66**
verify **57**, **66**
verifyMatch **57**, **66**
specific properties
Algorithm **55**, **64**
AlgorithmList **55**, **64**
BIR **55**, **64**
CapPrematchData **56**, **64**
CapRawSensorData **56**, **64**
CapRealTimeData **56**, **64**
CapSensorColor **56**, **65**
CapSensorOrientation **56**, **65**
CapSensorType **56**, **65**
CapTemplateAdaptation **56**, **65**
RawSensorData **52**, **56**, **59**, **65**
RealTimeDataEnabled **56**, **65**
SensorBPP **56**, **65**
SensorColor **56**, **65**
SensorHeight **56**, **65**
SensorOrientation **56**, **65**
SensorType **56**, **65**
SensorWidth **56**, **65**
State property **55**, **64**
StatusUpdateEvent **57**, **66**
implementation notes for **63**
Streaming Fingerprints **24**
Streaming Fingerprints (.NET API) **35**
Streaming Fingerprints (ActiveX) **46**

U

UpdateFirmware method **56**, **65**
UpdateStatistics method **57**, **65**
UPOS common properties
See common properties

V

verify method **57**, **66**
verifyMatch method **57**, **66**

W

Wavelet Scalar Quantization (WSQ) Compression **26**

CROSSMATCH

Corporate Headquarters:

Crossmatch
3950 RCA Boulevard, Suite 5001
Palm Beach Gardens, FL 33410 USA

www.crossmatch.com